TECHWOLF

Hi, we're TechWolf.

We extract the data and build the AI agents that help the largest companies in the world understand their people and the work they do.

CENEKA HACKATHON 2026 · WORKSHOP

Warre · Researcher **Ruben** · Software Engineer **Hanne** · Technical Team Lead



TODAY'S SPEAKERS

Three of us from the pack



Hanne

Technical Team Lead



Ruben

Software Engineer



Warre

Researcher

THIS MORNING

Six parts before you start building

01
TechWolf
Warre

02
Starting a hackathon
Ruben

03
Building in parallel with AI
Ruben

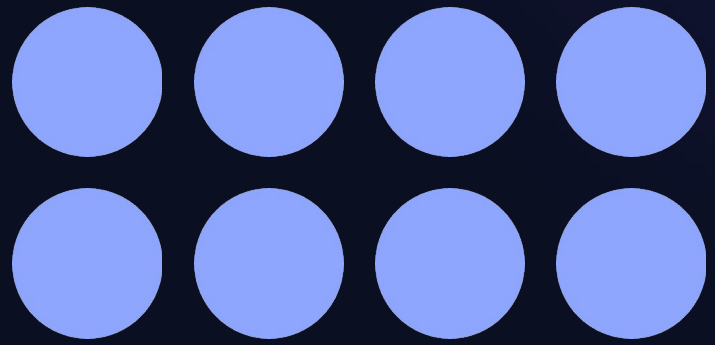
04
Data and ML in practice
Warre

05
Cost and scale
Ruben

06
Tips, the case and Q&A
Warre

2017 · 2018 · 2019–2021 · 2022–2024 · 2026

It started as a course project



8 STUDENTS · DESIGN PROJECT · GHENT UNIVERSITY

Dispatch: matching students to jobs, by skills



Then a real hackathon: first place

Design Project

↓

Dispatch prototype

↓

★ 1st at Delen Private Bank hackathon



2017 · 2018 · 2019–2021 · 2022–2024 · 2026

Founded, and the first pivot

Student jobs

→

Skills in companies

SEPTEMBER 2018 · TECHWOLF BV IS FOUNDED



2017 2018 ● 2019–2021 2022–2024 2026

The scrappy years

2 first hires, 2019

€1M raised in lockdown, 2020

CUSTOMERS SIGNED

9 in the first 3 years

10 in 3 months



THE BIRDHOUSE, GHENT

2017 2018 2019–2021 ● 2022–2024 2026

Scaling up

2022
€10M
Series A

2023
London
First office abroad

2024
$43M
Series B

2024
New York
First US office





2017 · 2018 · 2019–2021 · 2022–2024 · 2026

2026: the age of agents

250k+
HSBC employees in one skills taxonomy

Agents
Our first own agents shipped

SF
Next office: San Francisco

WHERE WE WORK

Four offices, one pack

Ghent	HQ
London	2023
New York	2024
San Francisco	NEXT



CULTURE

Five principles



Run with the pack



Aim for the Moon



Build a Rocket Ship



Open Up



Care Deeply

8 YEARS, THE WHOLE PACK

Same pack, a lot bigger

















THE QUESTION EVERY COMPANY ASKS

What work do we do, and who can do it?

HOW CUSTOMERS USE TECHWOLF

Our data, inside every agent

AGENTS THEY ALREADY USE

- Copilot
- ChatGPT
- Claude



OUR OWN AGENTS

- Analyst Agent
- Job Architecture

ONE QUESTION, THREE DATA SOURCES

Asking the Analyst Agent

HR LEAD

“Which customer service roles will AI change most, and what do we train for?”

- **Work:** tasks AI will change
- **Skills:** what people have
- **Market:** what others hire for
- **Answer**, with the evidence

WHAT THE AGENTS KNOW

Three kinds of intelligence, one data layer



Skills

Which skills do we have, and which do we need?



Work

Which tasks happen here, and which will AI change?



Market

How do we compare to everyone else?

2B+ JOB POSTINGS

UNDER THE HOOD

From raw signals to answers



WHY WE HACK

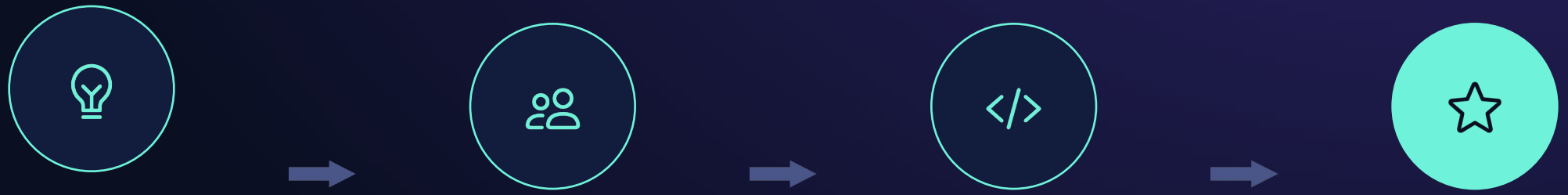
TechWolf started at a hackathon.

Remember this photo?



OUR YEARLY HACKATHON

Two days, the whole company



Idea Marketplace

anyone pitches, all vote

Teams form

3–5, around an idea

Build

mixed teams

Demo & awards

named after our values

WHY WE KEEP DOING IT

What two days of hacking gives us

Everyone builds
Researchers debug TypeScript

Titles disappear
The VP fixes merge conflicts

Process takes a break
Push to main before the demo

Real products
Work Intelligence: 2024 hackathon

AFTER THE DEMO

From prototype to roadmap



Show it

whole company

Test it

with customers

Invest in it

a task force

PART 02 · RUBEN

Starting a hackathon

Idea → Scope → Architecture

Doing a hackathon in the age of AI

TECHWOLF

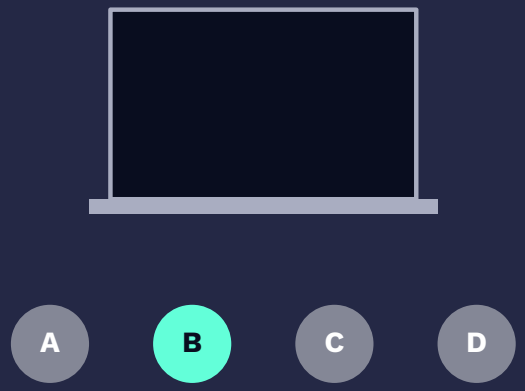
4 people, 6 hours



TECHWOLF

Don’t fall in the trap

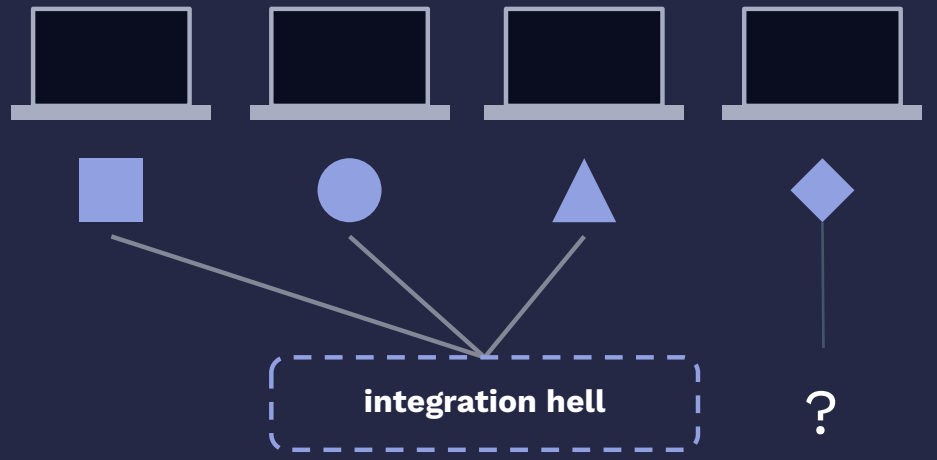
1 · ALL ON ONE LAPTOP



1 person builds, 3 watch

Nothing runs in parallel

2 · ALL ON DIFFERENT LAPTOPS



4 parts that don't fit together

Nothing works at the end

Tasks are too granular

BEFORE AI

Define tasks and divide them

task 1	A	task 2	B
task 3	C	task 4	D
task 5	A	task 6	B
task 7	C	task 8	D

Tasks change faster than you can assign them

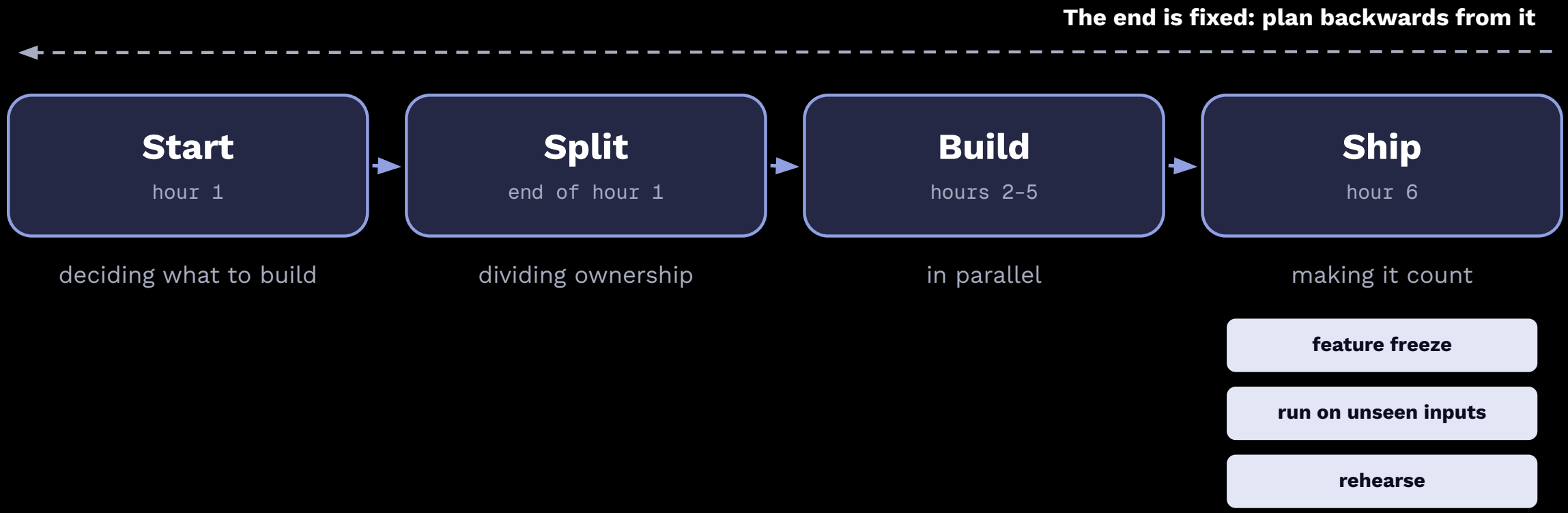
NOW

Each person owns a slice

slice 1	A	**slice 2**	B
owned end to end		owned end to end	
slice 3	C	**slice 4**	D
owned end to end		owned end to end	

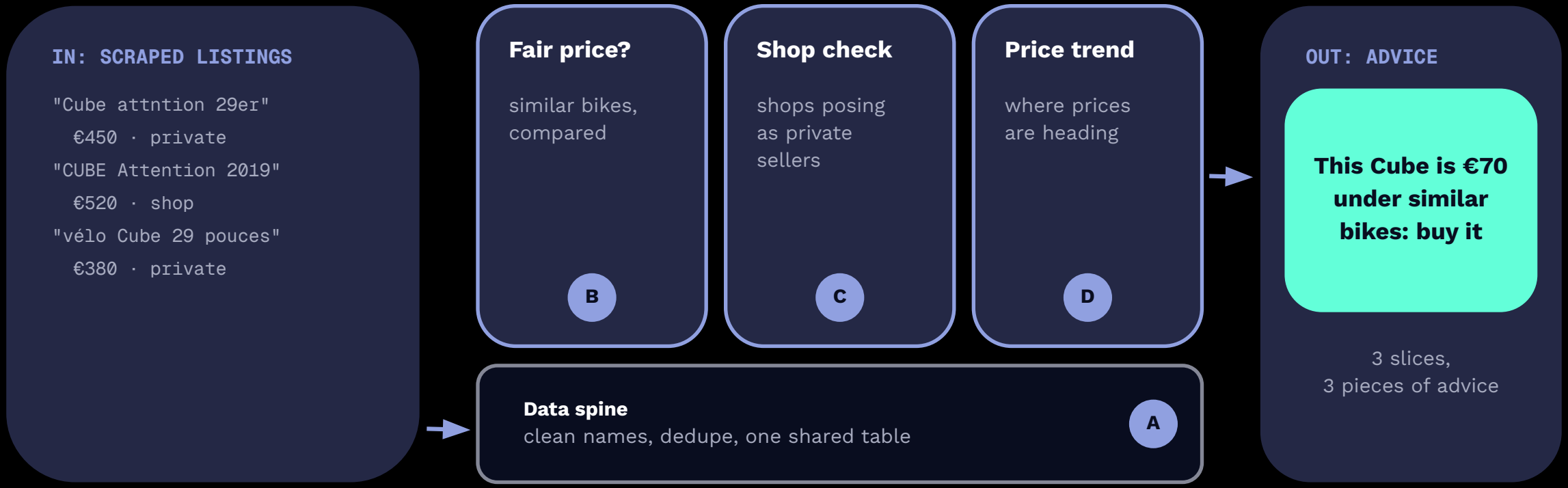
Spend your thinking on what to build

Your 6 hours in 4 phases



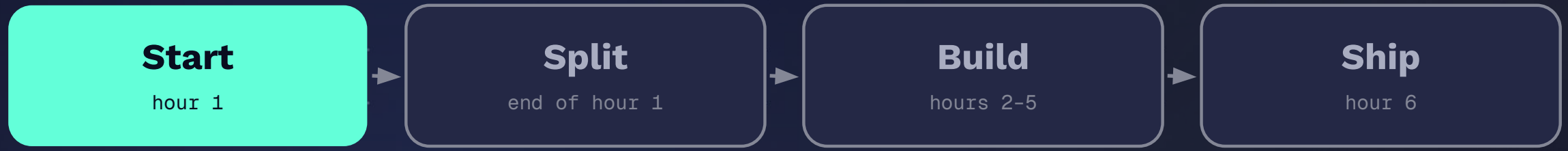
Running example: a price checker for second-hand bikes

Same shape as this afternoon, different data



PHASE 1 OF 4 · HOUR 1

Start, deciding what to build



A dashboard isn't a solution: end with advice



most teams stop here

Check every idea against what the jury scores

Would a customer pay for it?

value

Is it cheap to run?

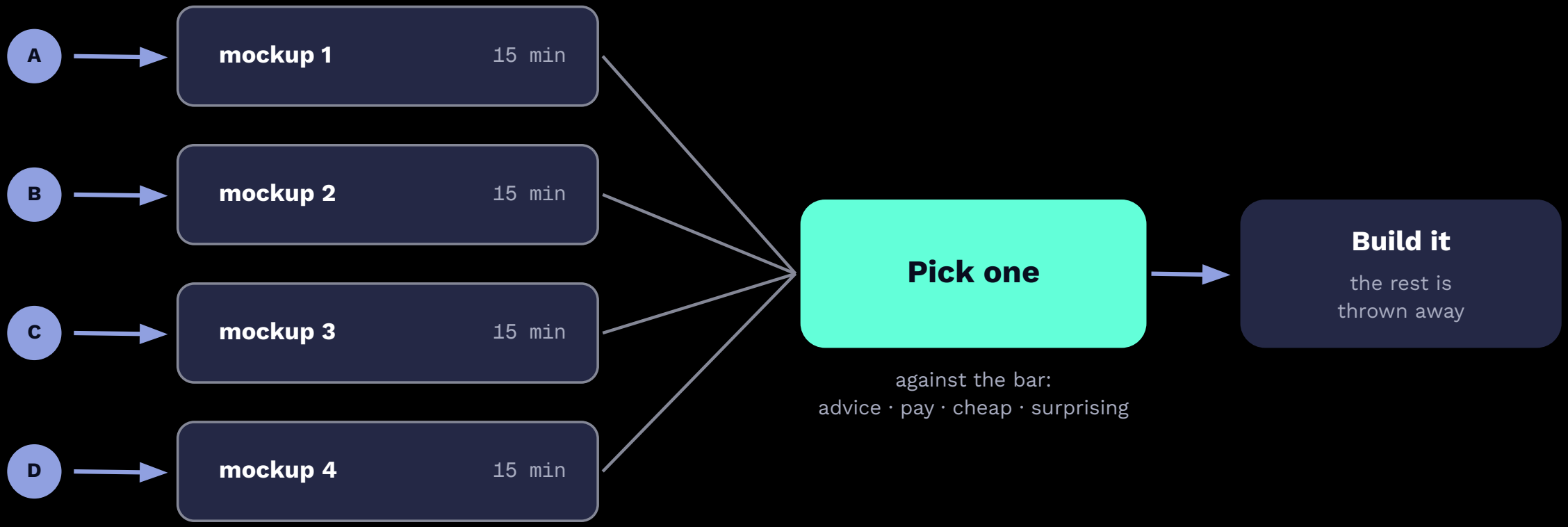
cost and scale

Is it surprising?

creativity

Prototype instead of discussing

Each person (or agent) vibe-codes a throwaway mockup of one idea in 15 minutes



Be ambitious, within reason

1 user

1 question

1 demo path

NOT-DOING LIST

mobile app

user accounts

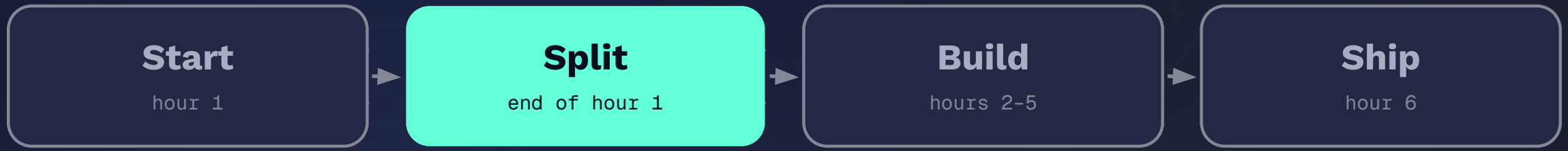
live scraping

Write it down.
Otherwise someone ends up building random stuff.

The jury gives you inputs you've never seen: generalise rather than hardcode the demo case.

PHASE 2 OF 4 · END OF HOUR 1

Split, dividing ownership



How to divide work efficiently?

HORIZONTAL: LAYERS

- **Frontend** A B C D
- **Backend** A B C D
- **Database** A B C D

everyone waits on everyone

VERTICAL: SLICES ON A THIN SPINE

Product shell one page, a card slot per slice — A

Fair price? — B

`model`

embeddings for similar bikes

`data → card → advice`

Shop check — C

`model`

LLM labels a sample, small classifier the rest

`data → card → advice`

Price trend — D

`model`

plain statistics, no LLM needed

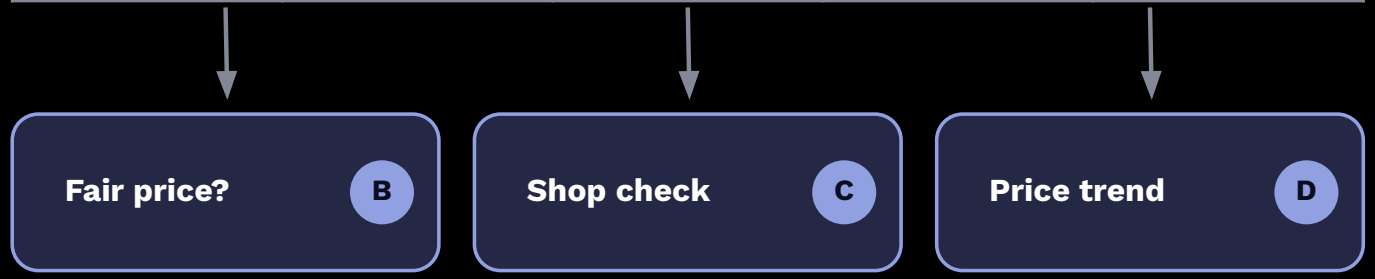
`data → card → advice`

Data spine clean names, dedupe, one shared table — A

Contracts first: one table in, one card out

SPINE TABLE · WRITTEN BY A, READ BY EVERY SLICE

listing_id	brand	bike_type	seller	price_eur
L-001	cube	mtb	private	450
L-002	trek	road	shop	300



CARD · READ BY THE SHELL

```
{
  "slice": "fair_price",
  "headline": "€70 under
     similar bikes",
  "advice": "buy it",
  "worth_eur": 70
}
```

stub table + stub card, day one

Agree on both before splitting up.
With stubs, nobody waits on anyone.

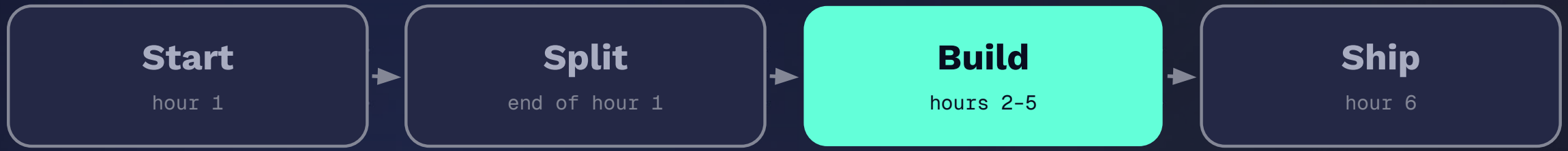
The spec is the source of truth

Read specs, not code

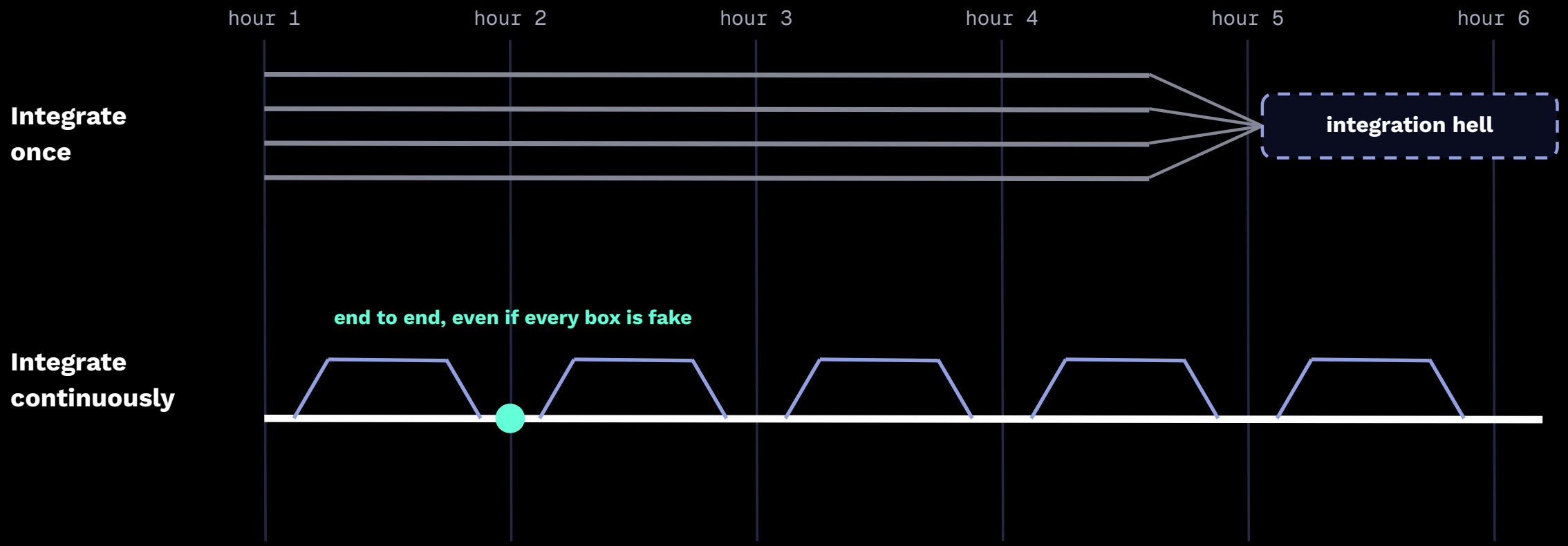


PHASE 3 OF 4 · HOURS 2-5

Build, in parallel

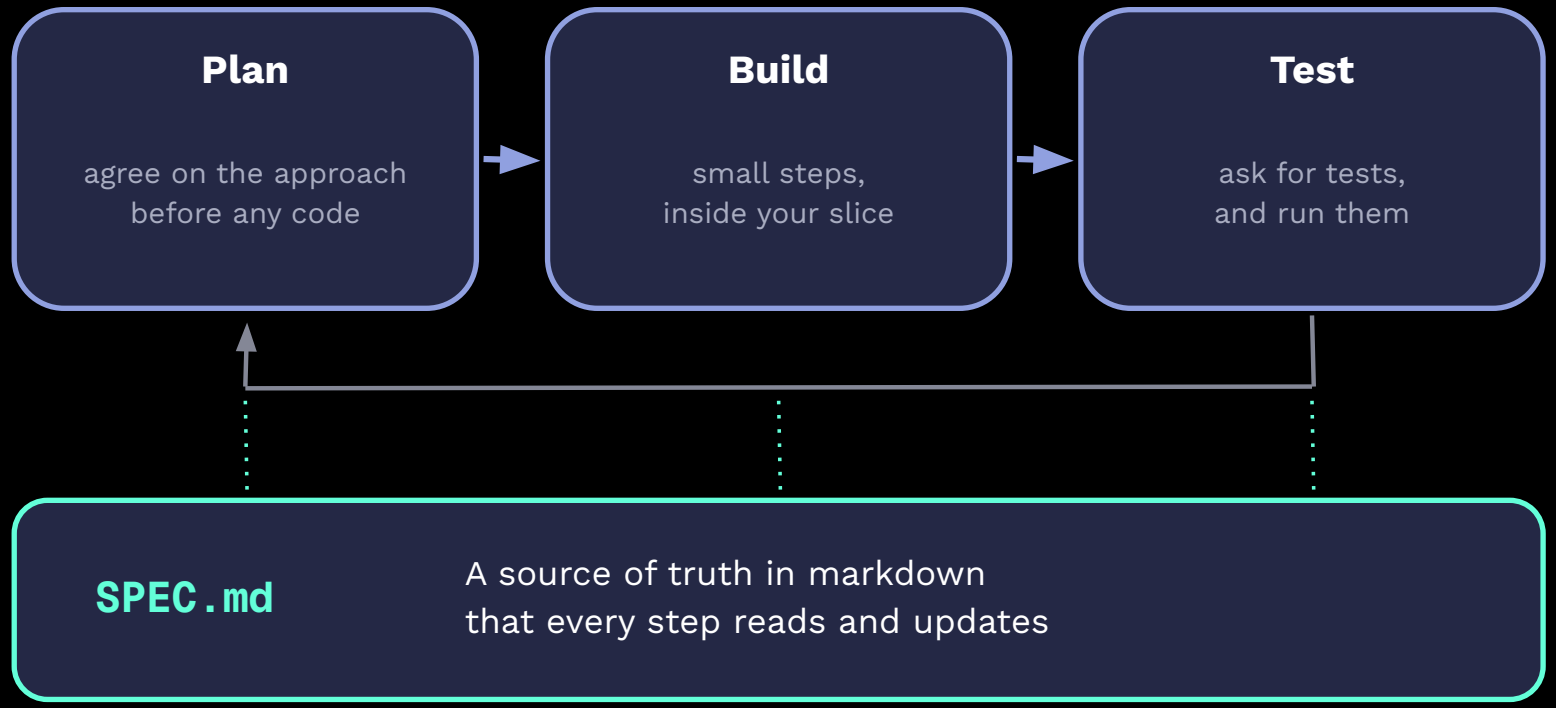


One repo, integrate early and often



Have an agent work just to check whether everything still works

Working well with an AI coding tool



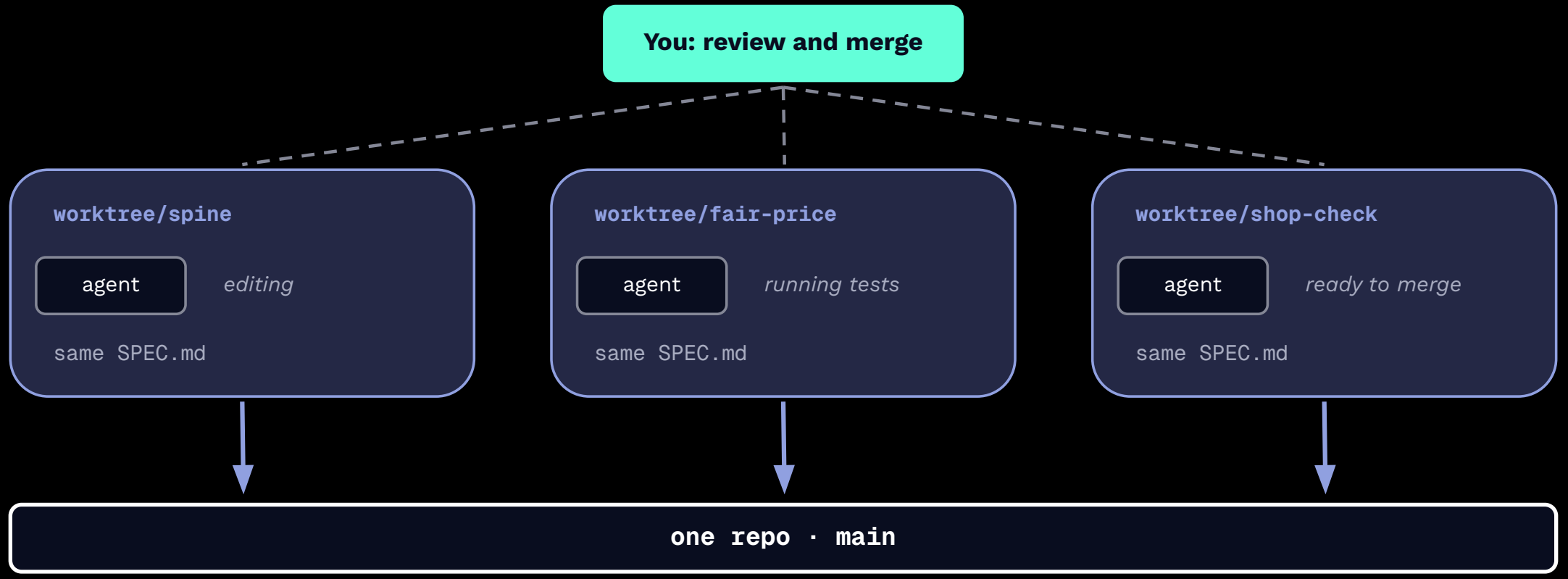
Keep the context small and focused

`the whole repo`

`your slice + spec`

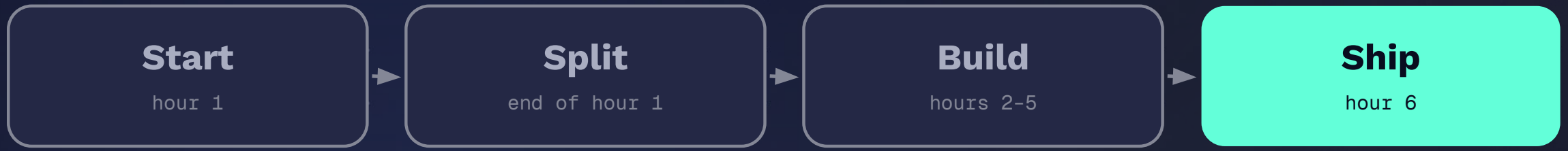
less to read, fewer wrong guesses

You’re a manager now



PHASE 4 OF 4 · HOUR 6

Ship, making it count



"Done" isn't done

Verification is the hard part: run end-to-end tests on known examples

"All done!
Every test passes."

your agent, 16u42

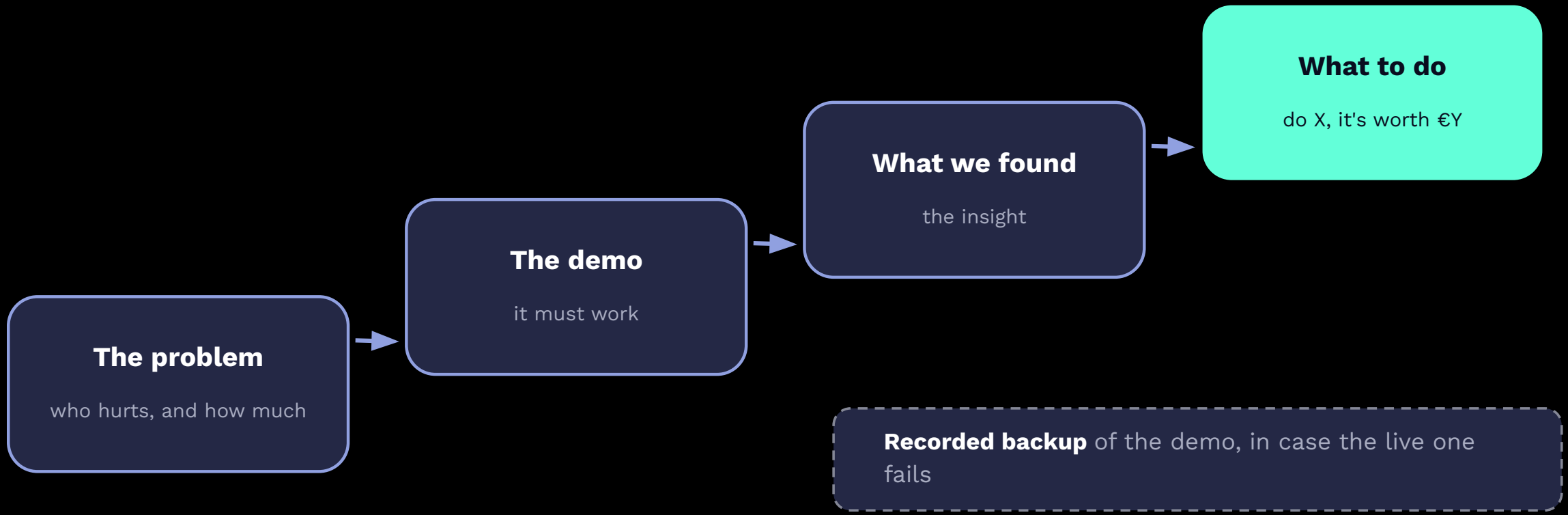
known example	expected	got	check
Cube Attention 2019	€480–560	€520	ok
Trek FX 3 (same shop, 3×)	1 listing	3 listings	fails
vélo Cube 29 pouces	mtb	unknown	fails

Think about the edge cases

a duplicate shop

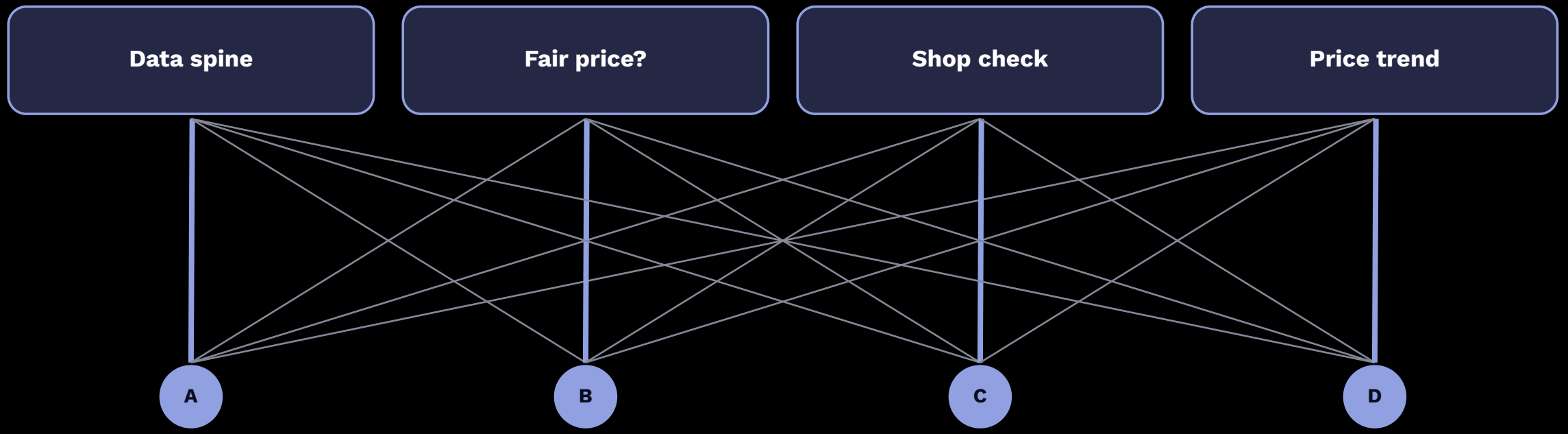
a listing in another language

Tell a story that ends in advice



Every team member can explain every part

The jury will ask



own one part, explain all four

Takeaway freeze early, rehearse, and make sure everyone can explain everything.

The afternoon on one slide

1 · hour 1

Start

One goal that ends in advice, picked by prototyping, plus a not-doing list

2 · end of hour 1

Split

Each slice has an owner; slices meet at one table and one card format

3 · hours 2–5

Build

Something end to end by hour 2, and integrate every hour after that

4 · hour 6

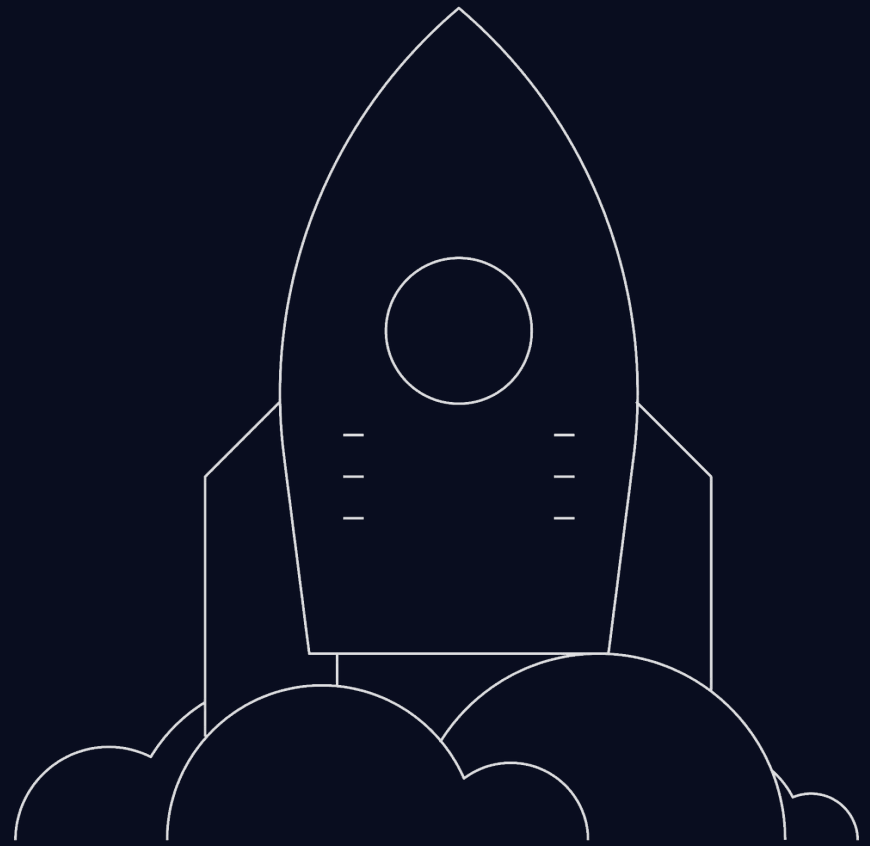
Ship

Freeze early, rehearse, and make sure everyone can explain everything

PART 04 · WARRE

Data and ML in practice

From messy rows to a model you can defend



WHY MACHINE LEARNING AT ALL

Where rules run out

Beyond keywords → same idea, other words or languages

Too big/messy for rules → learn from examples instead

Predict what's next → churn, demand, risk from history

Hidden structure → groups and outliers nobody labelled

Match and rank → the best fit out of millions

Generate and act → write text and code, run agents

If you can write the rule, write the rule. Use ML where you can only show examples.

Train a deep network only when you really have to



STEP 1

Rules and SQL

dedup, filters, joins, counts, a regex for the obvious

free, instant, explainable



STEP 2

Open models, off the shelf

embeddings, a Hugging Face classifier, an LLM with a prompt

no training, one GPU pass



STEP 3

Learn from your data

logistic regression or LightGBM on embeddings, trained on labels (an LLM made?)

minutes to train, runs on a laptop



STEP 4

Train a deep model

only when steps 1 to 3 measurably fall short

hours, GPUs, a real test set

THE METHOD FOR THIS AFTERNOON

Five steps



1 · Understand the data

what is in a row, what hides in it



2 · Lock the use case

then list what you need from the data



3 · Map it on paper

real inputs and outputs per step



4 · Dumbest PoC first

thin, end to end, then add brains where it pays



5 · Measure, iterate

baseline, a metric that means value, a walled test set

Read the data before you model it

- **Every column at a glance**
  nulls, distinct values and ranges per field
- **20 random rows**
  read in full, out loud, as a team
- **Search for your signal**
  is it there, and how do people write it?
- **Follow one company**
  coverage across time and company dimensions

```
-- read 20 random rows, really read them
SELECT * FROM 'data.parquet'
USING SAMPLE 20;
-- is your signal there, and how is it written?
SELECT title, description
FROM 'data.parquet'
WHERE description ILIKE '%machine learning%';
-- follow one company from start to finish
SELECT date, company, title FROM 'data.parquet'
WHERE company ILIKE '%rabobank%' ORDER BY date;
```

DuckDB runs SQL straight on a Parquet file: pip install duckdb. For every column at a glance: SUMMARIZE SELECT * FROM 'data.parquet';

One row, three kinds of information

ONE RAW ROW, AS POSTED

Northlight Software, an **Amsterdam**-based company that builds scheduling tools for small businesses, is looking for a **Marketing Coordinator** to plan and run our email, social, and content campaigns. You have 1-3 years of marketing experience, clear writing skills, and a knack for turning campaign data into practical next steps. We offer a salary of **€38,000-€45,000**, 25 days of annual leave, and a yearly learning budget. *Northlight is an equal opportunity employer, and we welcome applicants of all backgrounds and will gladly make reasonable adjustments during the application process. To apply, send your CV and a short cover letter to jobs@northlight.example by 20 November 2026.*

Explicit (bold)
company, city, title, salary range: a regex or a lookup gets these

Hidden (underlined)
seniority, skills, language, company type, sector: needs a model or a prompt

Noise (italic)
legal boilerplate, application instructions, deadlines: strip before you count or embed

Then: how do entities connect? company × time × location × title. That is where the use cases are.

Six spellings, one entity. Or two?

- UGent
- Universiteit Gent
- Ghent University
- UGent Faculteit Ingenieurswetenschappen
- Universiteit Gent & UZ Gent
- Hays, recruiting for Ghent University



Ghent University
the first four are one entity: clean, block, match, check

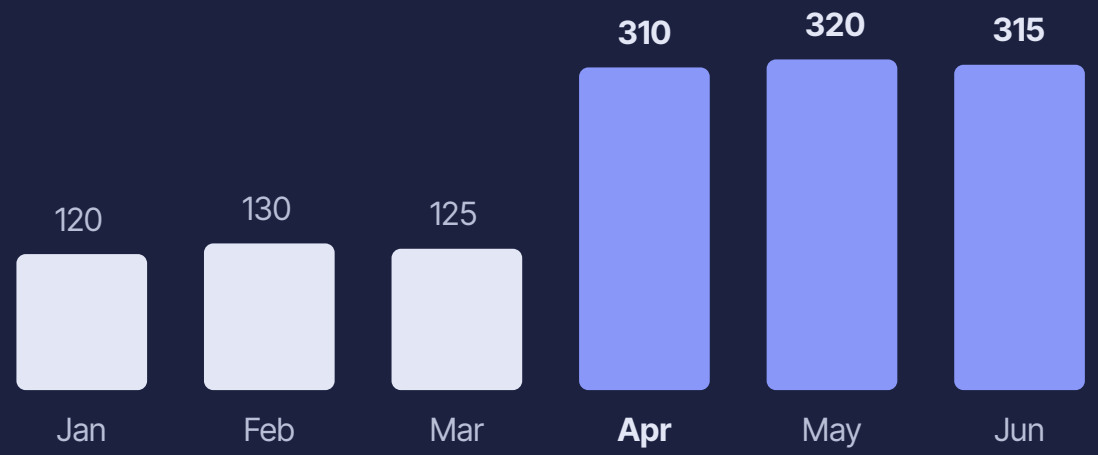
Joint name: decide the rule
one entity, two, or a link between them? Write the decision down

Middleman: its own type
drop it, keep it as “agency”, or recover the client from the text

What are all available entities?

Hidden variables shape what you observe

VACANCIES PER MONTH, ONE COMPANY (SCHEMATIC)



April: a second job site got scraped. Hiring did not change, coverage did.

Ask what else moves the data

sources added or lost, site redesigns, reposts, seasonality, boilerplate

Shares, not raw counts

a share within the same source and period survives a coverage jump

Check against the outside world

if nobody else sees your curve, suspect your pipeline first

Lock the use case, then list what you need

WRITE IT IN ONE SENTENCE

"**[Person]** uses **[output]** to decide **[X]**."

A model without a decision attached is a hobby.

Then list what you need

per piece of information: explicit, hidden or missing? and what bends it?

FOR EACH PIECE OF INFORMATION, HOW DOES IT OCCUR?

Question	What you will find
Format	free text, a number, a date; a different format per source and language
Missing	nulls, but also placeholders: "N/A", "-", 0, 1900-01-01, "see description"
Duplicates	the same ad on two sites, reposted every month, with small edits
Noise	boilerplate, typos, mixed languages, HTML left in the text
Over time	coverage jumps, a field that changes meaning, a source that disappears
Hidden variables	who posted it, where it was scraped, when; do they bend what you measure?

Draw it on paper first, with real inputs and outputs

RAW ROWS (TITLE, TEXT, COMPANY AS POSTED, DATE) GO IN ON THE LEFT

Normalise company

in: "UGent Fac. Ir."
out: ghent-university, type = employer
needs: cleaning and a fuzzy match

Tag each ad

in: the full text
out: role family, seniority, on-topic yes/no
needs: intelligence, the only step that does

Aggregate

in: tagged rows
out: one row per company and quarter, as shares
needs: a GROUP BY

Advice

in: that table
out: three actions, each with a number
needs: a template, or an LLM on 20 numbers

A real example next to every arrow shows which steps need intelligence. Here: one.

Per step: the dumbest thing that gives the right output

Transformation	Dumbest PoC (minutes, no model)	Upgrade, only once measured short
Normalise company	lowercase, strip legal suffixes, exact match	fuzzy match, then name embeddings for the long tail
On-topic yes/no	a keyword list on the title	embeddings + logistic regression on LLM-made labels
Role family	a regex on the title	title embedding, nearest label in a taxonomy
Seniority	"senior|junior|lead" and "N years"	a small classifier on the text
Advice text	a template over the top 3 numbers	an LLM that reads 20 numbers, never a million rows

Design towards quick value
a thin pipeline, end to end, ugly, on a small sample

Then ask what is still missing
now you can see which step actually limits the result

Two questions before you add intelligence

For what real-world constraints am I optimising? Am I creating **sustainable** value?



What do I really need? Am I adding deep learning because it sounds fancy, or because it is actually needed?



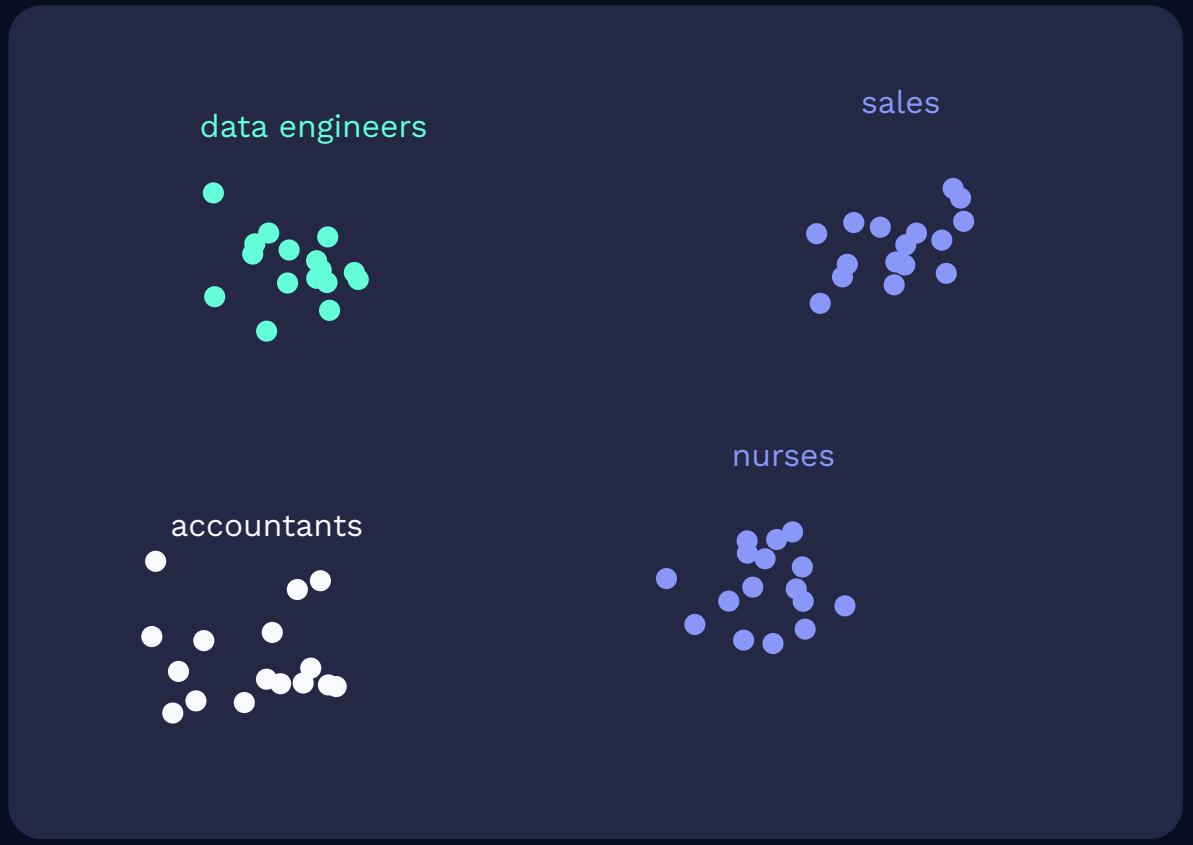
You don’t need to generate to understand

Task	Generative: an LLM writes the answer	Representation: embed once, compare
Is this ad about AI?	prompt every row, parse the reply	embeddings + a classifier, milliseconds per row
Which occupation?	ask for a label, hope it exists in ESCO	nearest ESCO label vector, always a valid label
Same company?	ask about every pair of names	name vectors, cosine similarity, one threshold
Who are the peers?	ask it to list the competitors	average vectors per company, rank by similarity
Which new roles appear?	ask it to invent categories	cluster, then an LLM names each cluster from 10 rows

Generate once, at the end
the advice a person reads; before that, it is all meaning

Why the right column wins
one pass per row, deterministic, reusable for everything

Embeddings turn text into points you can work with



Search

nearest neighbours: find every posting like this one

Group

clustering: discover the roles nobody labelled

Compare

average the vectors per organisation, then compare those

multilingual-e5 or sentence-transformers: “boekhouder”, “comptable”, “accountant” land together

Small models often win, and a big model can teach them

Small models often win: trained on good labels, they match a big model on a clear task, at a fraction of the cost. So let the big model teach.



1 · Sample
spread over languages, years, sources



2 · LLM teaches
labels your rows, or generates new examples



3 · You check
100 labels by hand, fix the prompt, relabel



4 · Train small
embeddings + a classifier, minutes on a laptop

Hsieh et al. (2023): a 770M student beat the 540B model it learned from.
Bucher and Martini (2024): fine-tuned small models beat zero-shot LLMs at text classification.
TechWolf (Decorte et al., 2023): skill extraction trained on LLM-generated data gained 15 to 25 points.

Iterate and optimise!

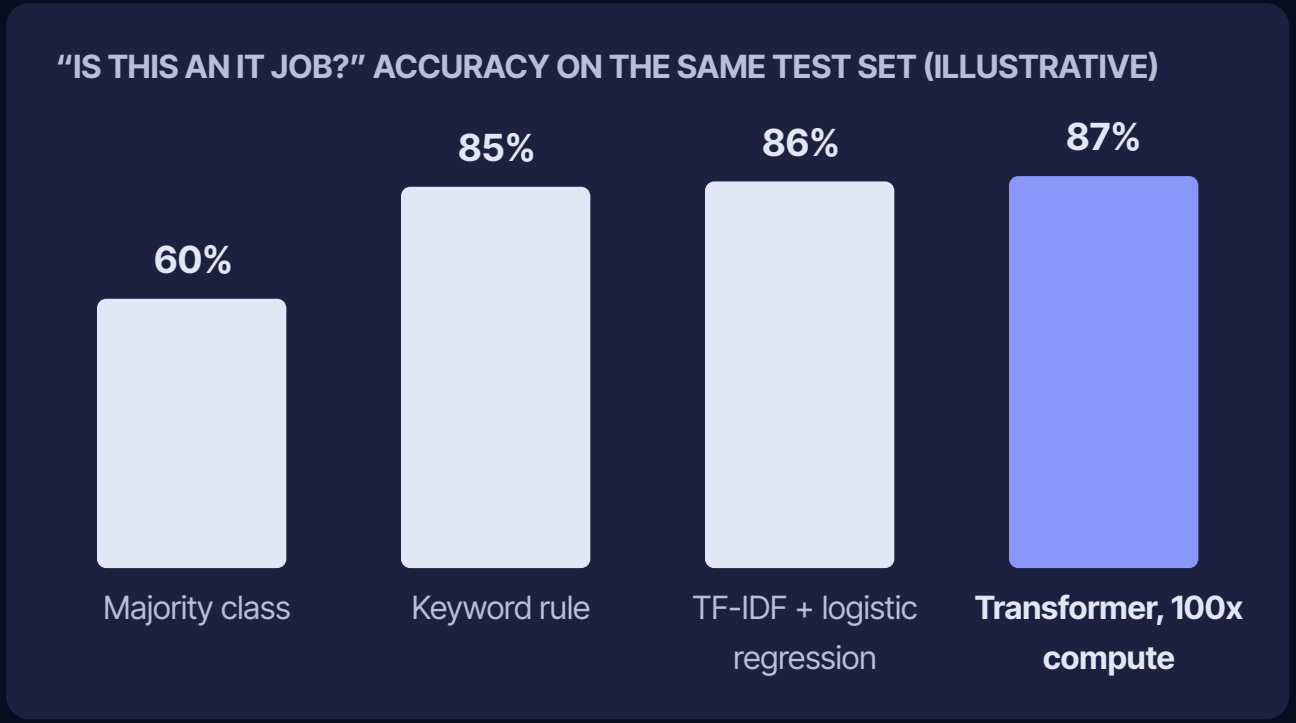


You will stir the pile. The next slides make every stir mean something.

- **A baseline** so a score has something to beat
- **A metric** that means value, not a benchmark
- **A wall** around the test set
- **A look inside**: what did it actually learn?

xkcd 1838, CC BY-NC 2.5

Measure the dumbest approach first



You cannot beat what you did not measure

the transformer buys +2 points for 100x the compute. Worth it? At least now you can answer.

Three rows, always first

DummyClassifier · one hand-written rule or SQL query · TF-IDF + logistic regression or default LightGBM

Is the answer literally written?

then a rule reads it. Add complexity for a reason, like in software.

Accuracy flatters rare classes

TRAINEESHIP DETECTOR, 3% POSITIVES (ILLUSTRATIVE)

0.97 **Accuracy** of "never say yes"

0.93 **ROC-AUC**, looks great

0.35 **PR-AUC**: mostly false alarms

A mean hides the rare class you care about. Print the class distribution next to every number.

What does this metric expose?

and what does it hide: rare classes, one language, one source, the worst slice

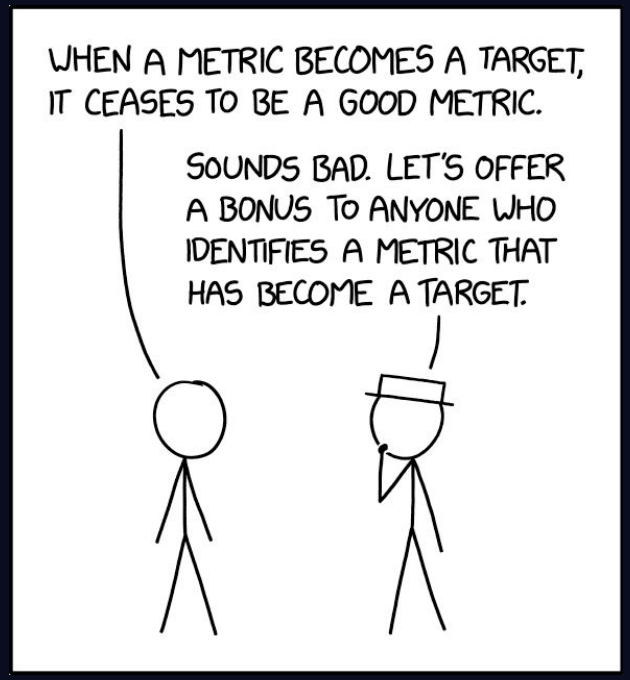
What is +1% worth in the decision?

fewer manual checks, a better advice, or nothing at all; write it down

Pick the metric for the class you care about

precision, recall and PR-AUC for the rare class; macro-F1 and per-class F1 for multiclass

Goodhart's law: the metric becomes the target



When I am optimising a metric, what does it mean to be 1% better? Am I benchmaxxing or creating value?

What does this metric expose, and hide?
rare classes, one language, one source, the worst slice: report per slice next to the mean

What is +1% worth in the decision?
fewer manual checks, better advice, or nothing at all; if you cannot say, stop optimising it

One target quietly becomes the wrong thing
when "higher is better" is all you check, you overfit to the metric, the test set and the demo

A wall around the test set

TRAIN

Fit on it

labels, few-shot examples, prompts: everything the model learns from comes from here, ids logged

VALIDATION

Overfit as much as you like

compare models, tune thresholds, iterate on prompt v3; it pays to treat it as if it were the test set

TEST

Look once, at the end

every time you peek and decide “better or not”, you leak. The number you report comes from here, with its n

Same distribution as deployment

the sources, languages and periods your system will really see, not the easy rows

No leaks through the back door

never in a prompt, no near-duplicates of train rows (MinHash), created after the LLM’s cutoff if you can

Read your 50 worst errors

THE 50 MOST CONFIDENT ERRORS, SORTED BY HAND (ILLUSTRATIVE)



Only 9 of 50 are a modelling problem. The rest is fixed in the data, the labels or the preprocessing, in an hour, for free.

Read input-output pairs

the 50 worst, the 20 most confident, 20 random; sort them by cause

Which features does it use, and why?

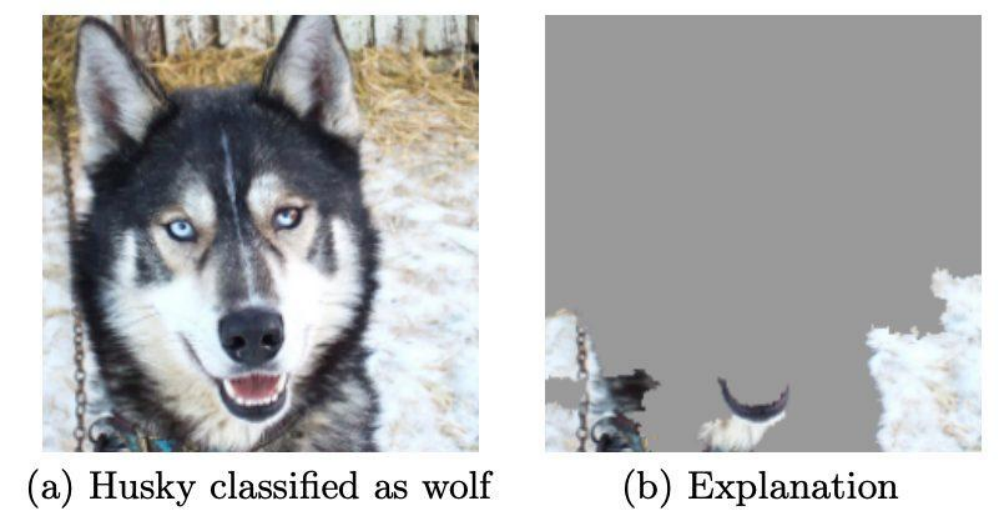
ablate a column, mask the company name, swap the language: does the prediction survive?

Hidden overfitting

great on one source, useless on another: the per-slice numbers show it before the jury does

What did it really learn?

What bias is in my data? Does the model really understand it, or did it find a shortcut?



(a) Husky classified as wolf · (b) the explanation: the model looked at the snow

Simplicity bias

a model learns the simplest way to separate the classes; if a shortcut exists in your data, it will take it

Find the snow in your data

the agency name, the language, the length of the text, one source: proxies that predict the label without understanding it

Ribeiro, Singh and Guestrin, "Why should I trust you?", 2016

A demo is not an evaluation

Salary extraction	The demo (hand-picked)	Random sample, seeded (n = 50 per slice)
Clean university posting	a perfect salary range	0% null; 3 successes and 3 failures shown
Agency posting	not shown	60% null: the worst slice
Overall	“it works!”	about 1 in 8 postings needs a manual check

3 + 3, seeded
three random successes and three random failures, not your favourites

Metric and n
next to every example, on the walled-off test set

The worst slice
in one line; where does it work well, where badly?

Translate it
“about 1 in 8 postings needs a manual check” beats “F1 = 0.87”

No labels is no excuse: four cheap quality checks

100 **Hand-check 100 rows**
96 random rows give precision within ±10 points at 95% confidence; split them over the team

κ **Two systems agree**
your small model vs the LLM, or two prompts; Cohen's kappa above 0.6 is substantial agreement

10 **Read 10 per cluster**
if you cannot name a cluster after 10 random members, it is not a cluster

 **Test against what you know**
the biggest organisations, the main cities, a trend from the news should show up; if not, suspect your pipeline

The jury will ask "how do you know this is correct?". One of these four is your answer.

$n = 1.96^2 \times 0.25 / 0.1^2 \approx 96$ rows; 400 rows give ±5 points

WHAT IT COSTS

Reading a million documents costs $5,000 or $3



List prices Oct 2026; batch APIs halve LLM prices; GPU: e5-small, ~500 docs/s on an A40

THE CHECKLIST THE JURY WILL USE

What you built is half the story

1 · DECIDE
Who uses the output, for which decision, and what is "good enough"?

2 · LOOK
Which name variants, middlemen and coverage jumps did you find, and what did you do with them?

3 · BASELINE
"Which part did you not build with AI, and why?" What did the dumbest approach score?

4 · LABEL
"Where did your training labels come from, and how noisy are they?"

5 · MEASURE
"How do you know this is correct?" A walled test set, the right metric, and the n.

6 · INTERPRET
What is the model actually using? Could that be a proxy for source, language or size?

7 · SCALE
"What happens with 100x more data tomorrow?" Cost per extra unit, runtime on the full data.

8 · TELL
3 random successes, 3 random failures, the worst slice, and "what did you try that did not work?"

One sentence each, with a number and an n, from any team member. Quoted questions are the jury's own.

PART 05 · RUBEN

Cost and scale

Cost and scalability is a feature

6/20

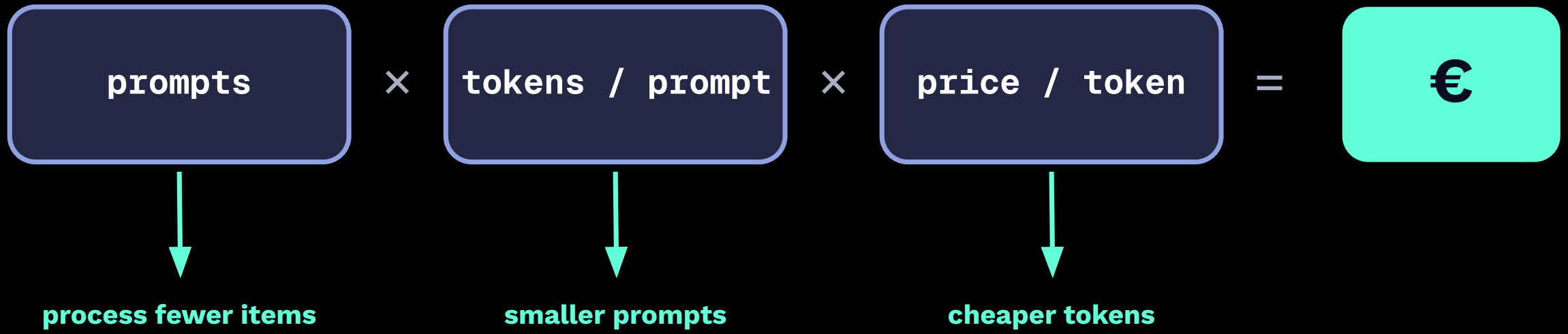
points for cost and scalability

100×

data size your solution should work on

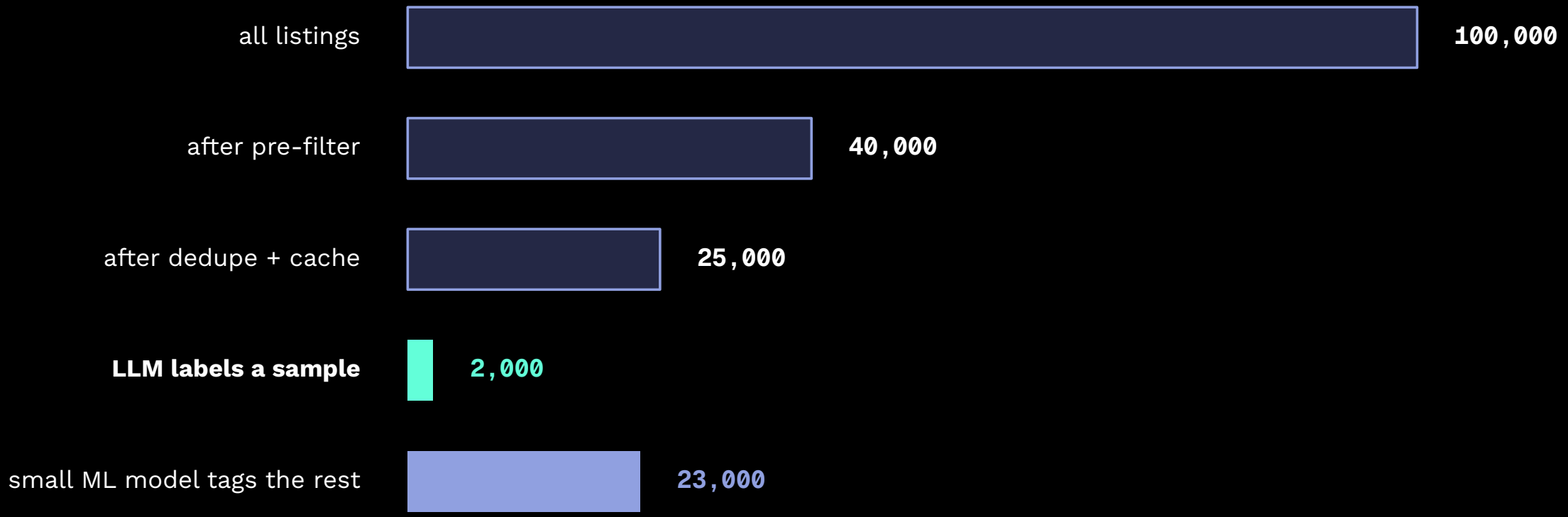
Tokenomics

Pull any one of them down



Items ↓ process fewer items

Illustrative numbers for 100k bike listings



LLM labels a sample, a cheap model handles the bulk: see Warre's talk

Tokens/item ↓ send only the fields you need

Illustrative token counts for one bike listing

title	15
description	1,200 → 300
seller_bio	250
photo_captions	300
price	5
location	10
posted_at	10



TOKENS PER LISTING

everything 1,790

what you need 320

~5× fewer tokens per listing

Measure on 100 real rows, don't guess

Price/token ↓ pay less per token

1

Cheaper model

A small model for the bulk; the big one only where it pays off

2

Prompt caching

Reuse the same long instructions across calls; cached tokens cost less

3

Batch API

Not in a hurry? Send it as a batch: cheaper, but results come later

Decision models: choose, don't generate

Pick from options you define and get a probability per option, e.g. Jev by TypeSafe

LLM: GENERATE TEXT

Is this seller a shop?



```
"Looking at the listing, the seller
has 14 other bikes for sale, so I
think this is probably a shop."
```

Parse the text, pay for every output token, and there's no probability to act on

DECISION MODEL: PICK AN OPTION

```
"seller": {"type": "choice",
  "criteria": {"shop": "...",
               "private": "..."}}
```



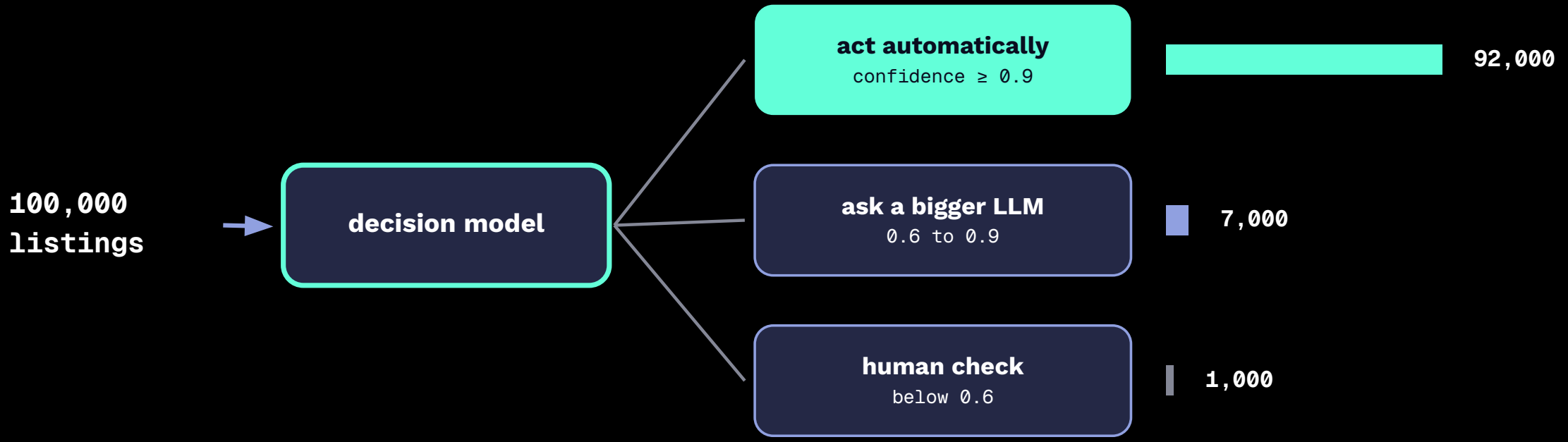
```
{"choice": "shop",
 "probabilities": {"shop": 0.91, "private": 0.09}}
```

Jev: $0.042 per 1M input tokens, output tokens free, API only (vendor docs, Nov 2026)

Not a calculator: no counting, maths or dates. Keep options short and unambiguous.

Confidence decides what happens next

Accept when confident, escalate when unsure. Illustrative thresholds: tune them on your data.



The expensive model only sees the 7% the cheap one is unsure about

Open decision models run on your own GPU

Same idea as Jev, open weights: a 4B–12B model fits easily on one A40 (48 GB)

CLOSED API

Jev 1.13

77.1

capability

$0.032 / 1k decisions

vs

OPEN · 4B

Torchcast Decision

77.5

capability

$0.0049 / 1k decisions

OPEN · 4B

H2O-Lightning

75.0

capability

$0.0052 / 1k decisions

OPEN · 12B

Jeff-1.1-Medium

81.9

capability

$0.013 / 1k decisions

Scores: JevBench (benchmarkheaven.com/jev-models), method v1.8, 31 Oct 2026.
More models: Decision Index (huggingface.co/spaces/multimodalart/jev-decision-index), ~250 open models.

Not every model fits: 27–31B models need ~55–65 GB in bf16, more than one A40. Pick 4B–12B, or quantise.

Fine-tuned 4B models match Jev on JevBench, at about a sixth of the cost

Pick the cheapest model that's good enough

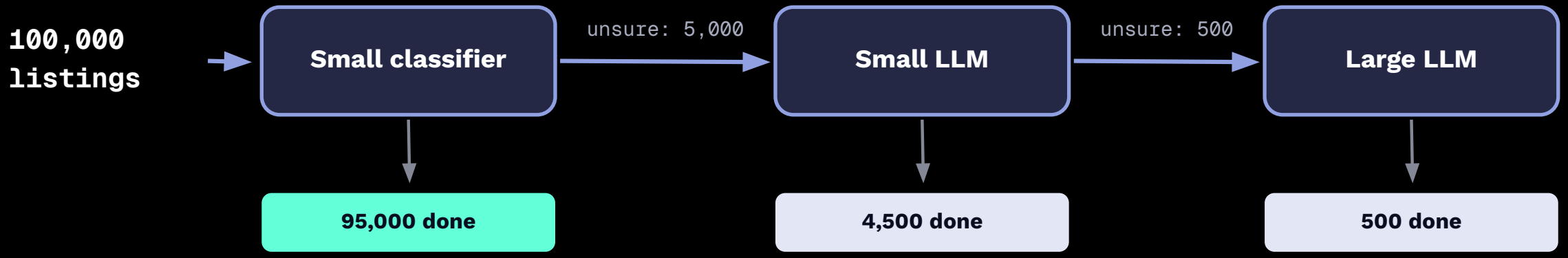
Start at the top; move down a rung only when the one above fails

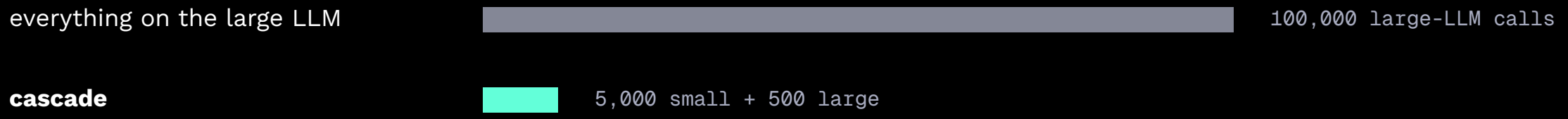
MODEL	COST	USE IT FOR (BIKES)
Rules / regex	free	price, year, frame size from the text
Small classifier	free	bike type, trained on LLM labels
Embeddings	€	similar bikes, duplicate listings
Small LLM	€€	shop or private, for the unclear listings
Large LLM	€€€	advice text for the final cards

LLM only where it pays off; a cheap model for the bulk

Cascade: escalate only when unsure

Illustrative numbers for 100k bike listings





It needs a confidence score: the classifier's probability, or let the LLM answer "unsure".

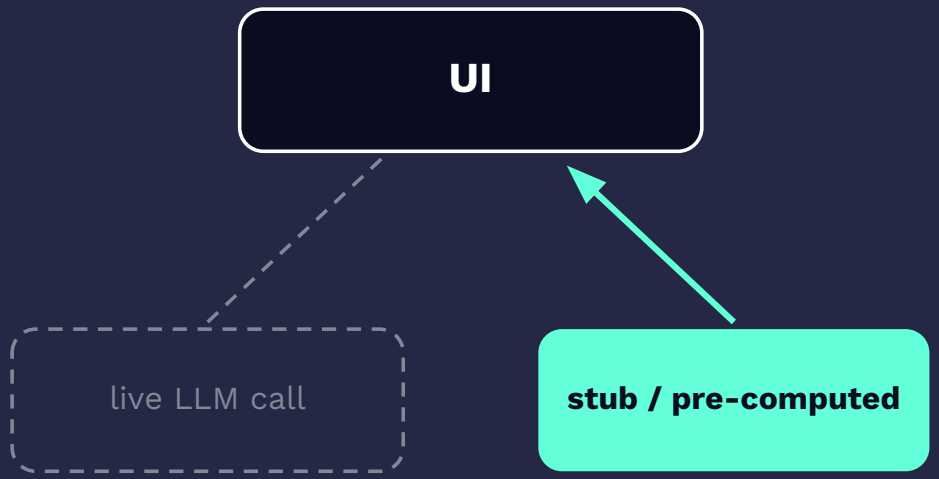
Bike listings: €450 becomes under €1

Same formula, one line per lever. Illustrative prices: check your provider's price page.

	items	tokens/item	€ per 1M tokens	batch	cost
naive	100,000	1,500	€3.00	–	**€450**
optimized	2,000	300	€0.30	× 0.5	**€0.09**
	LLM on a sample, small model for the rest	only the fields you need	cheaper model	batch API	

Spend on the product, not on iterating

WHILE YOU BUILD THE UI



Every reload of the page shouldn't cost money

PAIRWISE COMPARISONS GROW QUADRATICALLY



bars not to scale: 10,000× from top to bottom

Pre-select candidates first:
blocking or embeddings

What happens at 100×?

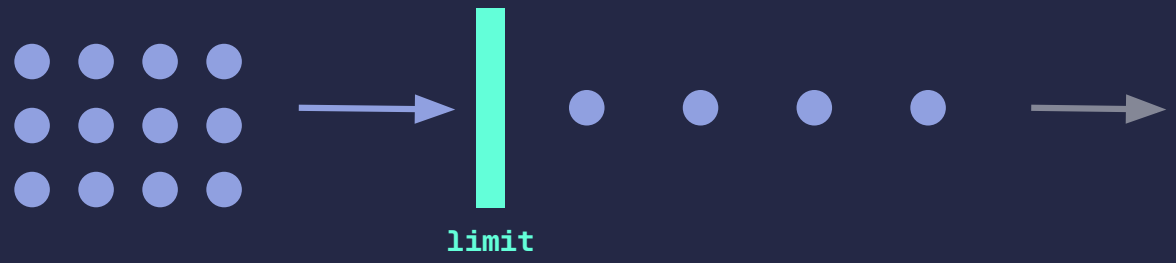
Assume the data is 100× larger. Some things just cost 100× more; others hit a wall.

	TODAY	AT 100×	
LLM cost	€0.09	**€9**	*just costs more*
API calls	100k · ~1.7 hours	**10M · ~7 days**	wall: **rate limits**
Data in RAM	1 GB	**100 GB**	wall: **memory**
Daily update	rerun 100k	**rerun 10M, every day**	wall: **incremental updates**
Checking output	50 by hand	**5,000 by hand**	wall: **validation**

Look for what hits a wall, not what just costs more

What breaks at 100×: API rate limits

Providers cap requests and tokens per minute: more data means more waiting



e.g. 1,000 requests per minute (illustrative)

today	100k calls	**≈ 1.7 hours**
100×	10M calls	**≈ 7 days**

Smaller models

higher limits and faster answers

Better pre-selection

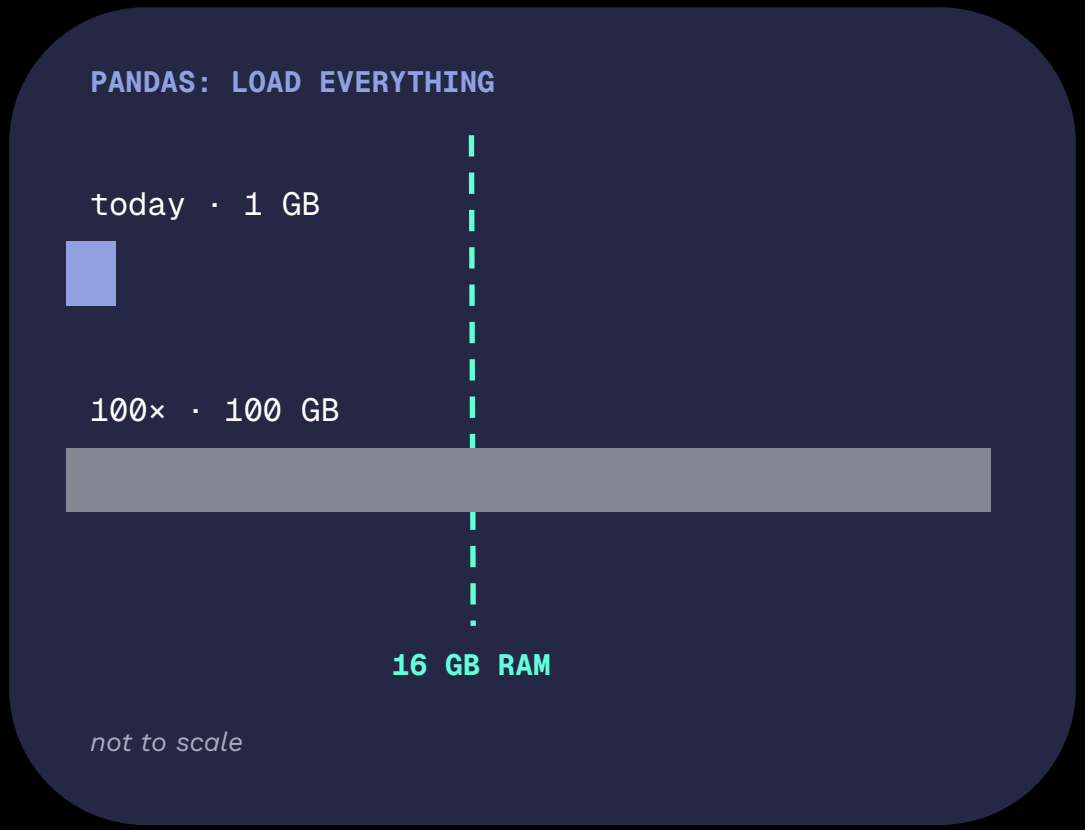
filter first, so fewer items need a call

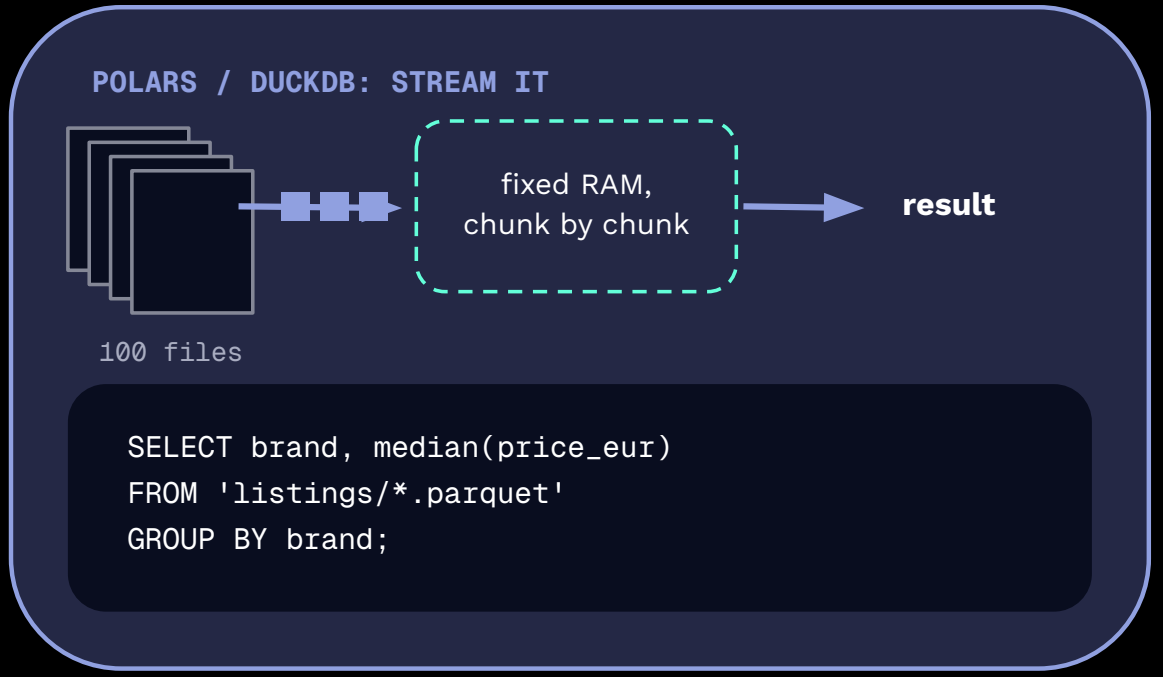
Distilled models

train a small model on the big model's answers; run it yourself, no limit

What breaks at 100×: memory

Pandas loads the whole dataset into RAM; at 100× it no longer fits





Bigger than one machine? Use a big data platform

What breaks at 100×: incremental updates

New listings arrive every day: reprocessing everything for each one doesn't scale

FULL RERUN

day 1

all 10M

day 2

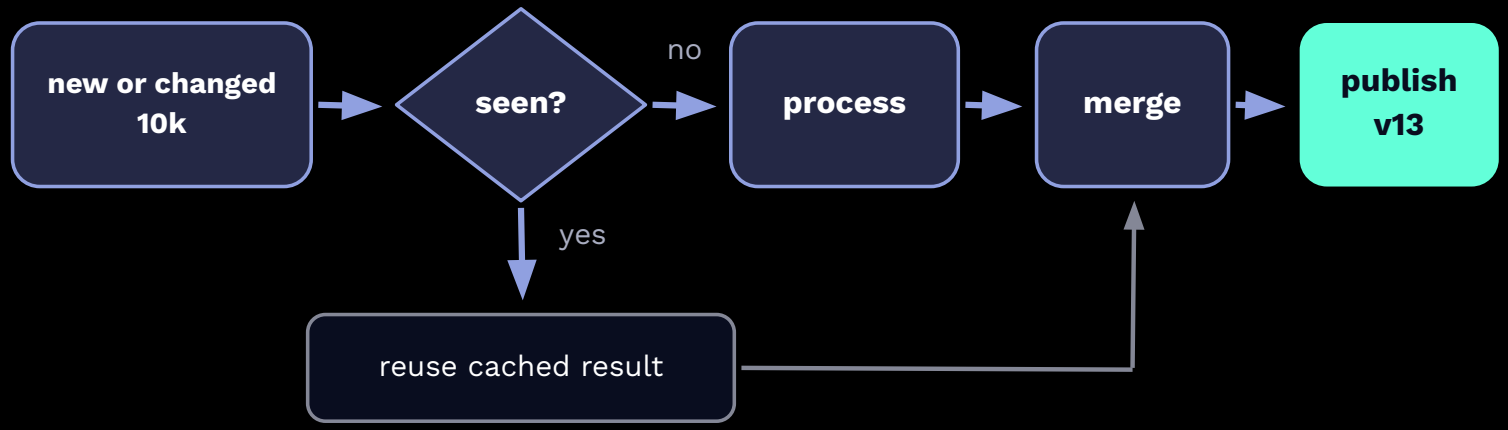
all 10M

day 3

all 10M

the same work again, every single day

INCREMENTAL



Key every item

by ID + a hash of its content

Cache the results

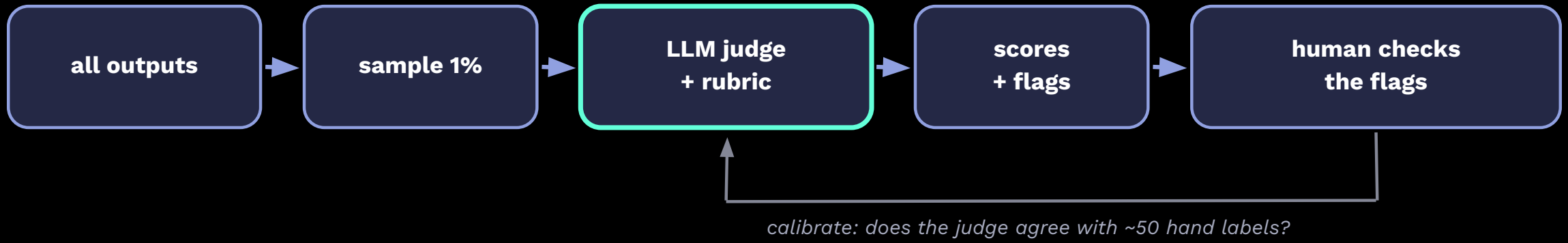
under that key

Ship versions

v12 → v13, with what changed

What breaks at 100×: validation

You can check 50 outputs by hand; at 100× nobody reads 5,000



RUBRIC · FAIR PRICE CARD

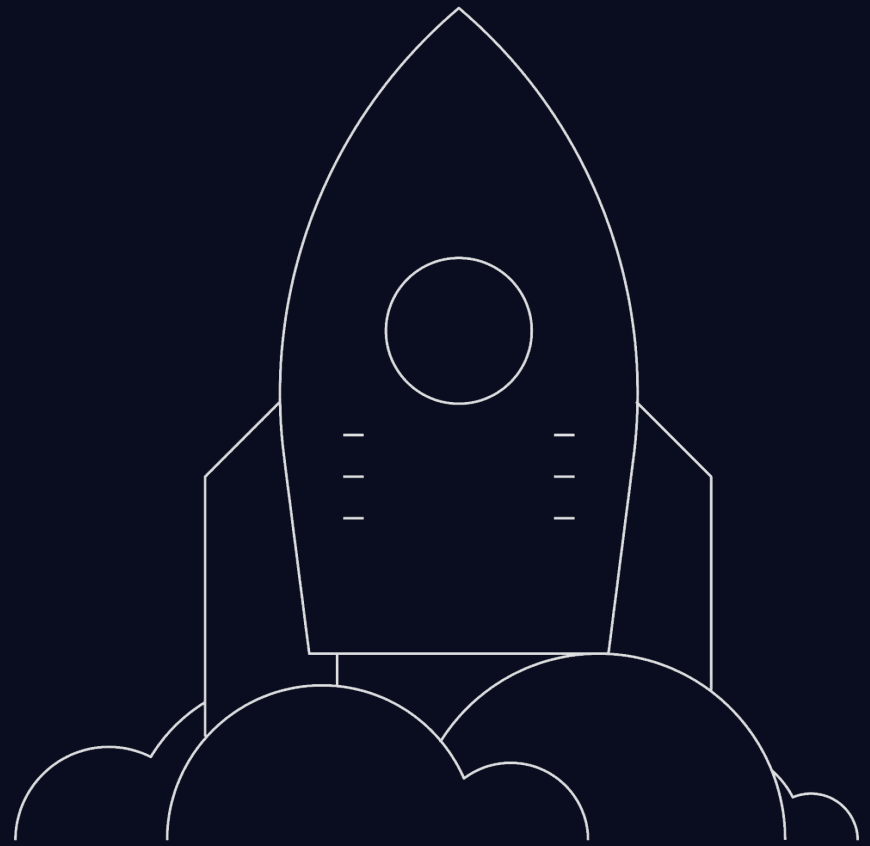
- compared to 5+ similar bikes?
- advice follows from the numbers?
- no invented bike models?

Keep the judge cheap

Judge a sample, not everything. A small model with a clear rubric is often enough.

PART 06 · WARRE

Tips from the jury table, then your challenge



TIPS FROM THE JURY TABLE

Demo one thing that works

One use case, end to end
a thin working path beats five half features

Running by half time
ugly but end to end, then improve the weakest step

Rough table early
on a small sample, so no lane waits on another

Freeze well before the end
the last stretch is for the full run, results, slides and rehearsal

Working demo in 90 s
start with the problem, then show it live

Rehearse, keep a backup
a 60 s screen recording saves you when the wifi dies

From the jury side, the teams that win show one thing that works, and know their numbers.

THE WORLD OF WORK MOVES FASTER

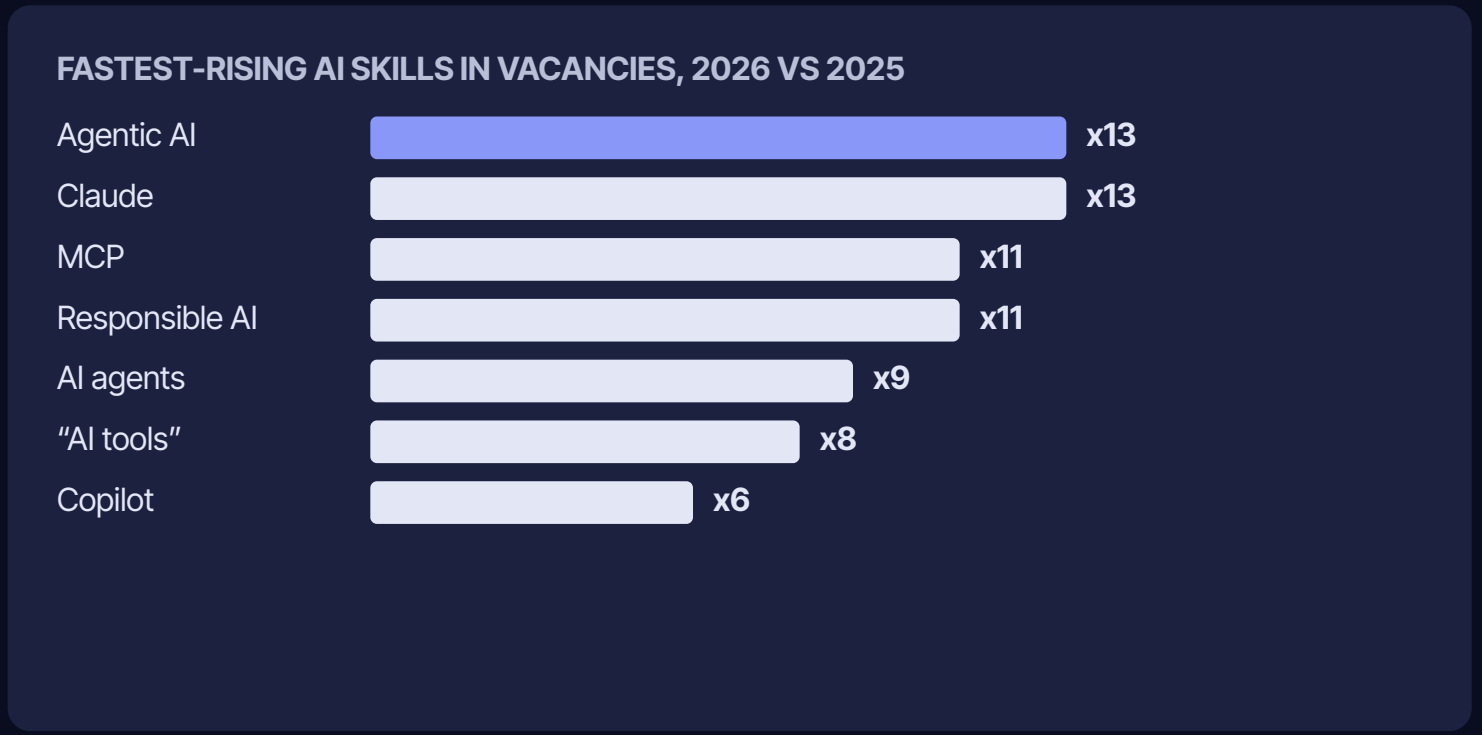
Skills that barely existed last year are in job ads today

x13

more vacancies ask for agentic AI in 2026 than in 2025

9x+

cheaper every year for the same AI performance



TechWolf data lake, share of vacancies per skill, Jan–Sep 2026 vs 2025 · price trend: Epoch AI (2025)

THE WORLD OF WORK MOVES FASTER

Every company asks: are we ahead or behind?

HOW FAST YOU CAN SEE WHAT A COMPETITOR DOES

Annual reports



a year late, and written for investors

Surveys and analyst reports



months late, a sample, self-reported

Job postings



every day, for every company, in detail

Bar length: rough delay before the information is public.

WHAT LEADERSHIP WANTS TO KNOW

Who are our real competitors for talent?

Are they moving to AI faster than we are?

Which of our roles and skills will change?

What does it cost us to catch up?

Nobody can answer these from the outside today.

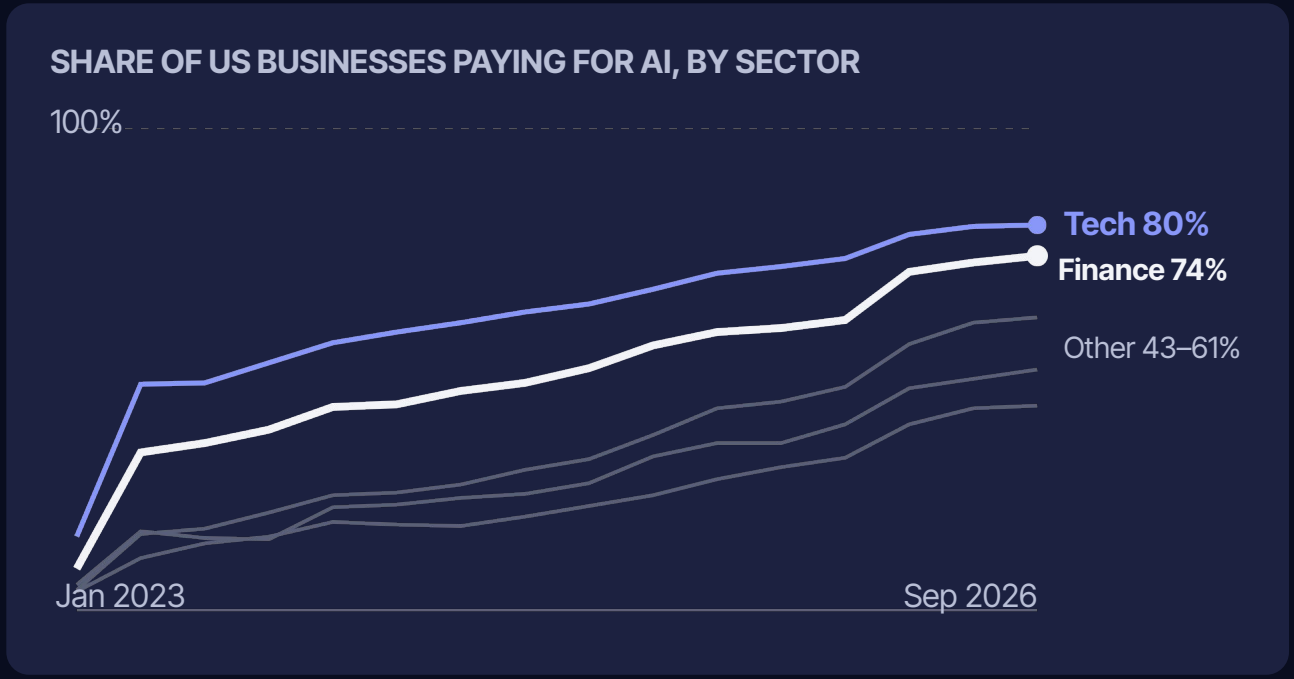
YOUR CHALLENGE

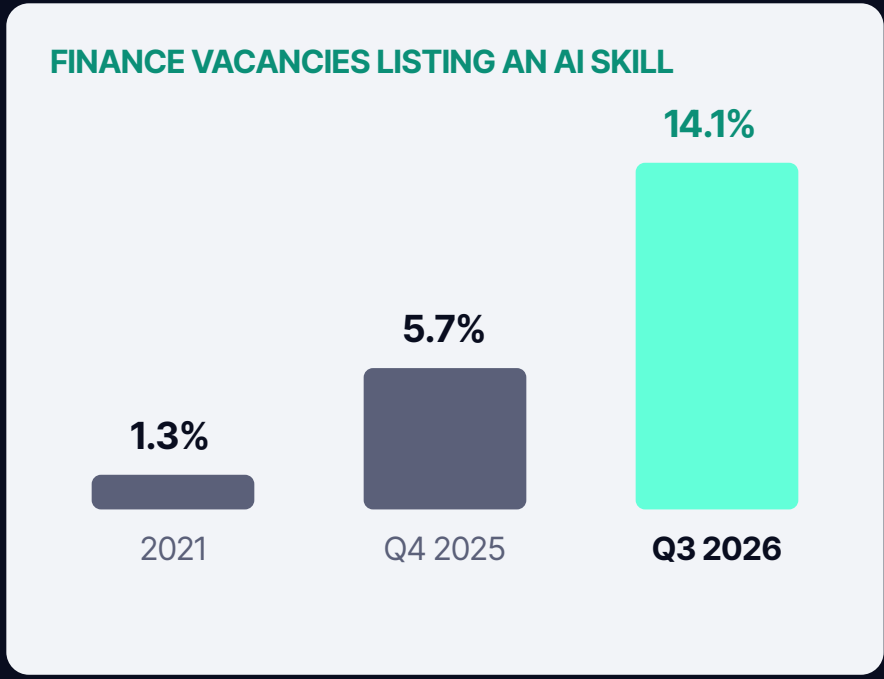
AI is changing how companies work and who they hire. Help a company understand where it stands.

Your playground: financial services.

WHY FINANCE

Finance buys AI like tech does, and now hires for it





Left: Ramp AI Index, Jan 2023–Sep 2026 · Right: TechWolf data lake, finance, Benelux + UK

WHY FINANCE

Why finance is the perfect place to look

74%

Racing
of finance firms pay for AI, second only to tech; 75% of UK financial firms use it

5,200

Restructuring now
jobs ABN AMRO cuts by 2028; ING cuts 1,250, mostly in anti-money laundering

4→21%

Peers drift apart
Citi's vacancies with an AI skill within one year, while Santander stays near 5%



Regulated
credit scoring and insurance pricing are high-risk under the EU AI Act, so AI governance skills rise too

If you can show a bank where it stands against its peers, you have built something people pay for.

Sources: Ramp AI Index · BoE/FCA 2024 · Retail Banker Intl · DutchNews · TechWolf data lake · EU AI Act

You get a million real vacancies, raw

1.5M

deduplicated job postings from finance companies and the recruiters that hire for them

Belgium, Netherlands, Luxembourg, UK

January 2021 to September 2026

WHAT IS IN EVERY ROW

- job title
- description
- company as posted
- date

WHAT IS NOT

- skills
- occupation
- industry
- clean company names
- competitors

Labels, names and peers are yours to build. That is where this morning pays off.

Parquet: a 20K-row sample to start on, and the full slice

WHAT YOU GET · 2

Plus a growing open data lake on AI and work

AI usage
who uses AI, and for which tasks
Anthropic Economic Index · OpenAI Signals · Microsoft Working with AI · Ramp AI Index

AI exposure per job
which jobs AI can take over or help
ILO GenAI exposure 2025 · GPTs are GPTs · Felten AIOE (per finance sub-industry)

Jobs and skills
one language for messy titles and skills
ESCO (NL, FR, EN) · O*NET · TechWolf JobBERT-v3 and skill models on Hugging Face

Labour market
outside numbers to check yours against
Indeed Hiring Lab and AI tracker · Eurostat · ONS · CBS · Statbel

Companies
who is who, and who owns whom
GLEIF (3.5M entities, parent links) · Companies House · KBO · ECB bank list

AI prices and tokens
what your solution costs, today and at 100x
LiteLLM and OpenRouter prices · Epoch AI · WildChat token counts

All free to use; most need no key or sign-up. Links, licences and local copies of the big files are in the README.

Synthetic data: only what a company would have inside

INSIDE THE COMPANY: ALLOWED

- ✓ its own AI usage logs: tokens, models, cost per team
- ✓ its own headcount per department and job
- ✓ its own job architecture or skills framework
- ✓ its own learning, training and recruiting records

OUTSIDE ITS WALLS: NOT ALLOWED

- ⚠ a competitor's AI spend or usage
- ⚠ a competitor's internal headcount
- ⚠ a ready-made list of who competes with whom
- ⚠ anything about named individuals

Rule of thumb: in the pitch, say who inside the company owns the data and how you would get it.

Your compute: a Claude budget and your own GPU

Resource	Use it for	Good to know
Claude key, hard quota	thinking: label a sample, hard cases, the advice	spend counts in the score
1 × A100, 40 GB	repeating: everything that touches every row	yours all afternoon, nothing to pay per token
Any encoder	embed, search, cluster: e5, bge-m3, JobBERT-v3	a base model embeds 1M postings in about an hour
Gemma 4 26B-A4B	an open LLM for prompts over many rows	Google, Apache 2.0, only 4B active per token: fast
Your own small model	classifiers trained on Claude's labels	logistic regression or SetFit: minutes to train

Gemma 4 fits in 8-bit
~28 GB of weights in 8-bit; BF16 (~52 GB) does not fit

Set it up in hour one
load your models while you plan, not at four o'clock

THE CHALLENGE

A few quick examples, to show the range

AI radar
how fast a bank's workforce moves to AI, next to its peers

Role shift
which roles shrink where AI skills appear

Skill gap
what you use AI for vs what the market hires for

Cost to catch up
hire, retrain or buy: what closing the gap costs

Examples, not the assignment. Start from the use case and the data, and find the angle that makes a company act. Whatever you build ends in concrete advice.

WHAT WE EXPECT

Hand in a working repo & slides, then pitch it

repo
zip file

Live demo
Is a bonus.

Architecture slide
components, which models where, and why

Cost slide
tokens, euro spent, euro per extra company, what breaks at 100x

Pitch
5 min + 5 min questions; everyone can explain every part

HOW WE SCORE

20 points, and cost counts as much as value

6

Solution value

concrete, correct advice a customer would pay for

6

Cost and scalability

LLM only where it pays off, a sane cost per company

4

Creativity

an unexpected insight, data source or approach

4

Pitch

clear story, demo works, on time, whole team explains

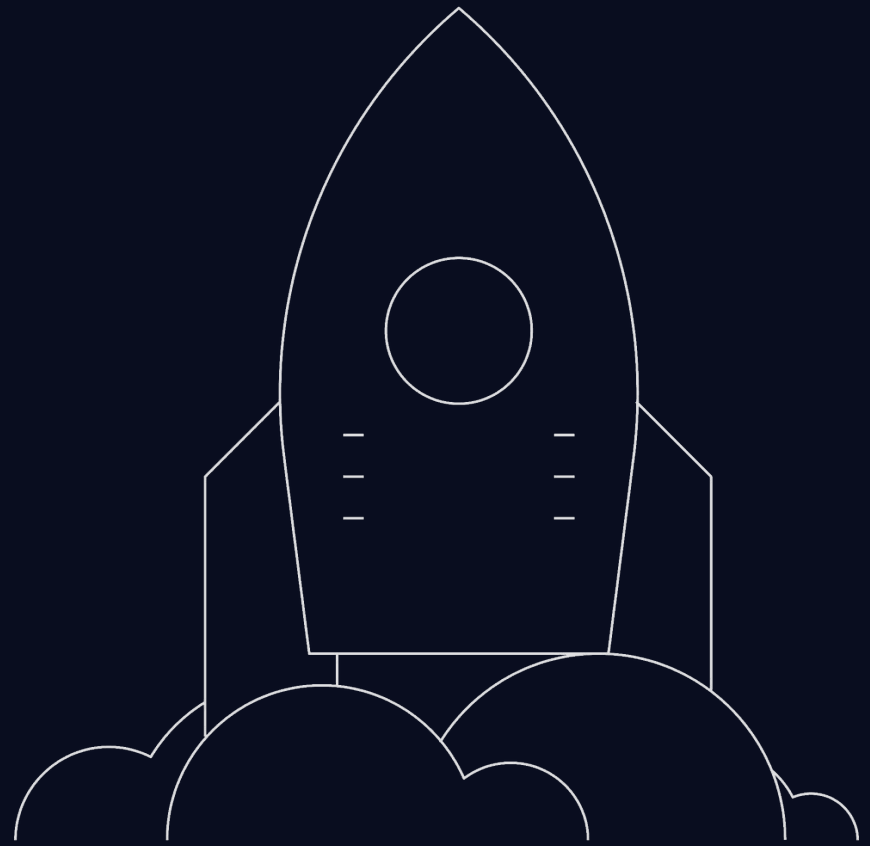
EXPECT 2 OR 3 QUESTIONS, EACH TO A DIFFERENT TEAM MEMBER

“Which part did you not build with AI, and why?” · “What happens with 100x more data tomorrow?” · “How do you know your competitor list is correct?”

Questions?

Lunch is thinking time · kit, data and keys right after

Mentors on the floor all afternoon



YOUR GPU MACHINE

Connect to your team's machine

1 × NVIDIA A100 (40 GB), 85 GB RAM, 12 CPU cores. Connect from the venue Wi-Fi or UGent eduroam only.
Terminal: run your SSH command, type yes, then your password · VS Code: Remote-SSH, F1, Connect to Host · Need sudo? Ask an organiser.

Team	SSH command	Password
team1	ssh team1@34.122.76.172	cenekathon-team1
team2	ssh team2@34.59.185.126	cenekathon-team2
team3	ssh team3@34.28.255.3	cenekathon-team3
team4	ssh team4@34.10.198.35	cenekathon-team4
team5	ssh team5@35.188.4.8	cenekathon-team5
team6	ssh team6@34.55.89.213	cenekathon-team6
team7	ssh team7@35.184.223.137	cenekathon-team7
team8	ssh team8@34.57.84.223	cenekathon-team8
team9	ssh team9@34.123.55.243	cenekathon-team9
team10	ssh team10@104.154.245.111	cenekathon-team10

Team	SSH command	Password
team11	ssh team11@34.7.81.66	cenekathon-team11
team12	ssh team12@136.112.136.128	cenekathon-team12
team13	ssh team13@34.171.191.64	cenekathon-team13
team14	ssh team14@34.132.159.25	cenekathon-team14
team15	ssh team15@34.122.184.214	cenekathon-team15
team16	ssh team16@34.42.166.217	cenekathon-team16
team17	ssh team17@34.29.182.134	cenekathon-team17
team18	ssh team18@136.65.157.123	cenekathon-team18
team19	ssh team19@34.29.79.243	cenekathon-team19
team20	ssh team20@34.60.59.235	cenekathon-team20